



Unikraft Cloud Platform

Enterprise Documentation

1 Legal Notice

Copyright © 2026, Unikraft GmbH. All rights reserved.

This software and related documentation ("Unikraft Software") are protected under relevant copyright laws. The information contained herein is confidential and proprietary to Unikraft GmbH ("Unikraft") and/or its licensors. Without the prior written permission of Unikraft GmbH and/or its licensors, any reproduction, modification, use or disclosure of Unikraft Software, and information contained herein, in whole or in part, shall be strictly prohibited.

BY OPENING THIS FILE, RECEIVER HEREBY UNEQUIVOCALLY ACKNOWLEDGES AND AGREES THAT THE SOFTWARE/FIRMWARE AND ITS DOCUMENTATIONS ("UNIKRAFT SOFTWARE") RECEIVED FROM UNIKRAFT AND/OR ITS REPRESENTATIVES ARE PROVIDED TO RECEIVER ON AN "AS-IS" BASIS ONLY. UNIKRAFT EXPRESSLY DISCLAIMS ANY AND ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NONINFRINGEMENT. NEITHER DOES UNIKRAFT PROVIDE ANY WARRANTY WHATSOEVER WITH RESPECT TO THE SOFTWARE OF ANY THIRD PARTY WHICH MAY BE USED BY, INCORPORATED IN, OR SUPPLIED WITH THE UNIKRAFT SOFTWARE, AND RECEIVER AGREES TO LOOK ONLY TO SUCH THIRD PARTY FOR ANY WARRANTY CLAIM RELATING THERETO. RECEIVER EXPRESSLY ACKNOWLEDGES THAT IT IS RECEIVER'S SOLE RESPONSIBILITY TO OBTAIN FROM ANY THIRD PARTY ALL PROPER LICENSES CONTAINED IN UNIKRAFT SOFTWARE. UNIKRAFT SHALL ALSO NOT BE RESPONSIBLE FOR ANY UNIKRAFT SOFTWARE RELEASES MADE TO RECEIVER'S SPECIFICATION OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM. RECEIVER'S SOLE AND EXCLUSIVE REMEDY AND UNIKRAFT'S ENTIRE AND CUMULATIVE LIABILITY WITH RESPECT TO THE UNIKRAFT SOFTWARE RELEASED HEREUNDER WILL BE, AT UNIKRAFT'S OPTION, TO REVISE OR REPLACE THE UNIKRAFT SOFTWARE AT ISSUE, OR REFUND ANY SOFTWARE LICENSE FEES OR SERVICE CHARGE PAID BY RECEIVER TO UNIKRAFT FOR SUCH UNIKRAFT SOFTWARE AT ISSUE.

2 Contact and Support

For more information, support, inquiries or to schedule a free consultation please contact:

Method	
Email	support@unikraft.com
General Enquiries	+1 (929) 333-3823
On-Call Support	+1 (717) 837-9955

Contents

1	Legal Notice	2
2	Contact and Support	2
3	Introduction	6
3.1	Platform Controller	7
3.2	Agent	7
3.3	VMM	7
3.4	Proxy	7
3.5	CoreDNS	7
4	Installation	8
4.1	Requirements	8
4.2	For cloud deployments, use cloud-init to automate this setup. See the Cloud Images section for examples.	11
4.3	Packages	11
4.4	Installation	13
4.5	Cloud Images	16
4.6	Client-side Tooling	19
4.7	CLI Configuration	20
5	Platform Startup	21
5.1	Systemd Targets	21
5.2	Startup Order	21
5.3	Services	22
5.4	Startup Troubleshooting	25
6	Platform Configuration	27
6.1	Managing Configuration	27
6.2	Configuration Sections	27
6.3	Applying Changes	31
7	Platform Controller	33
7.1	Initialization	33
7.2	Subsystems	33
7.3	Persistent Databases	35
7.4	User Management	36
7.5	Assertions	40
7.6	Autoscaling	40
7.7	Configuration	41
7.8	Flags Reference	41
8	Agent	52
8.1	Image Management	52
8.2	TLS Certificate Provisioning (ACME)	53
8.3	OpenTelemetry Metrics Export	54

8.4	API Log Proxying	54
8.5	Health checking	55
9	Virtual Machine Monitor (Firecracker)	56
9.1	Firecracker Integration	56
9.2	VM Lifecycle	56
9.3	Priority Boost	57
9.4	Startdata	57
9.5	Snapshots	57
9.6	Suspend and Resume	58
9.7	Console	58
9.8	ukpd Communication	59
9.9	Metrics	60
9.10	Logging	60
9.11	Key Flags In ukpd	60
10	Proxy	61
10.1	Architecture	61
10.2	OpenResty Layer	61
10.3	C Proxy Layer	62
10.4	Server-Timing Header	63
10.5	Instance UUID and Node ID Header	63
10.6	Service Groups	63
10.7	Certificate Management	64
10.8	Configuration	64
11	Snapshot Management	65
11.1	MTSS (Multi-Tier Snapshot System)	65
11.2	Snapshot Storage	67
11.3	Snapshot orchestration	67
11.4	Scale-to-Zero Wake-Up Flow	67
11.5	Scale-to-Zero Rate Limiting	68
12	Networking	69
12.1	Firewall	69
12.2	Dependencies	72
12.3	Network Isolation	73
12.4	TAP Interfaces	74
12.5	DNS Services	76
13	Volumes	82
13.1	Overview	82
13.2	Filesystem Types	82
13.3	Quotas	82
13.4	Storage Operations	83
13.5	Volume Storage	84
13.6	Volume Limits	84

13.7 Managed Volumes	84
13.8 Custom Filesystem Configuration	84
14 Images	86
14.1 Image Storage	86
14.2 Image Distribution Modes	86
14.3 Offline Image Import	86
14.4 Image References	87
15 TLS/SSL Certificates Management	89
15.1 Overview	89
15.2 Certificate Modes	89
15.3 update-ukp-certificates	92
15.4 Automatic Renewal	94
15.5 Renewal Hook	94
15.6 Switching Certificate Modes	94
15.7 Certificate Storage	95
16 Metrics	96
16.1 Architecture	96
16.2 Controller Metrics	96
16.3 Node-Level Metrics	97
17 Logging	98
17.1 Log Format	98
17.2 Log Path and Level	98
17.3 Log Rotation	99
17.4 Log Sinks	99
18 Health Checks	101
18.1 Overview	101
18.2 Agent Health Checks	101
18.3 Configuration	102
19 Troubleshooting	103
19.1 Diagnostic Tools	103
19.2 Stop Codes	105
19.3 Common Issues	105

3 Introduction

Unikraft is a platform for running applications as lightweight, hardware-isolated virtual machines with millisecond cold starts and stateful scale-to-zero. It replaces traditional container runtimes and orchestrators with a purpose-built stack that manages the full lifecycle of application instances — from image distribution and networking to autoscaling and observability — on a single physical host or across a cluster of hosts.

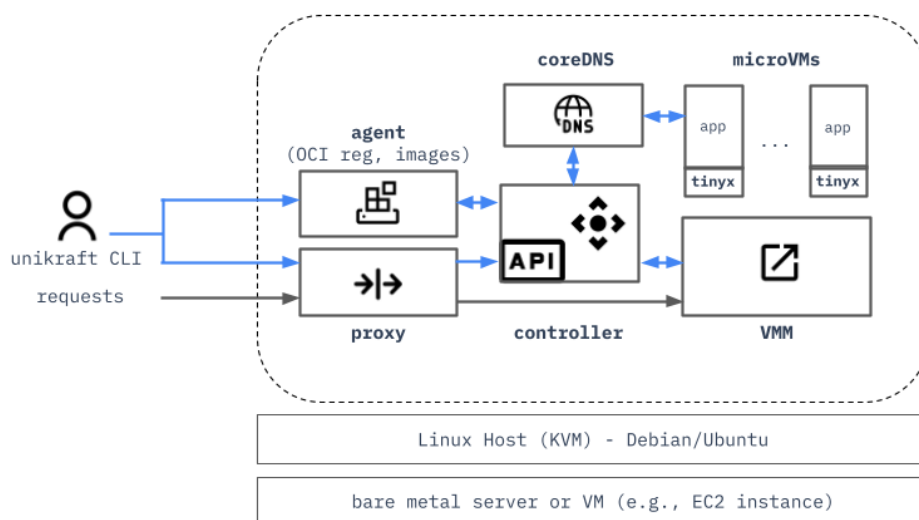


Figure 1: Architecture Overview

The platform is designed for operators who need strong multi-tenant isolation without the overhead of full virtual machines, and for workloads that benefit from fast startup times and scale-to-zero when idle. Each application runs inside its own MicroVM — a fully isolated hardware virtualization boundary with dedicated vCPUs, memory, and a network interface — providing the security properties of a virtual machine with the density and speed typically associated with containers.

Inside each MicroVM, the guest environment is provided by Tinyx, a minimal Linux-based init system (PID 1), and ukp-fuse, a FUSE daemon that serves the guest filesystem from the host. Together they replace a traditional container runtime with a purpose-built minimal stack that boots in under 10 ms. Tinyx handles guest boot and process supervision, scale-to-zero notifications, and filesystem mounting via ukp-fuse. ukp-fuse serves OCI image layers and read-only ROM objects to the guest.

This documentation covers the operator and infrastructure concerns specific to deploying and running Unikraft. User-facing features (instance lifecycle, templates, autoscale, volumes, etc.) are documented in the [public documentation](#).

Below, we describe each major platform component shown in the architecture diagram.

3.1 Platform Controller

The platform controller, `ukpd` (`ukp-platform`), is the central daemon on each host. It is written in C and is responsible for the entire platform control plane: VM lifecycle management, networking, storage, autoscaling, and the REST API that clients use to interact with the platform. `ukpd` manages six persistent databases, a worker pool for background operations, and a monitor subsystem that collects CPU metrics, VM health, and proxy traffic statistics used for autoscaling decisions. Every other component on the host — the agent, the proxy, `coreDNS` — depends on `ukpd` being running and healthy.

3.2 Agent

The agent (`ukp-agent`) is a sidecar process that runs alongside `ukpd` on each host. It handles tasks outside of the hot-path such as: OCI image pulling from external registries, ACME/TLS certificate provisioning via Let's Encrypt or a custom CA, Prometheus metrics export for node-level system metrics, and API log forwarding to an OpenTelemetry collector. The agent also runs an optional embedded OCI registry that allows developers to push images directly to the host without an external registry.

3.3 VMM

The Virtual Machine Monitor subsystem manages Firecracker-based microVMs. Firecracker (`ukp-firecracker`) is the hypervisor — each application instance runs as a separate Firecracker process managed by `ukpd`. The VMM subsystem handles the full VM lifecycle from boot to shutdown, startdata serialization (the per-VM configuration blob passed to the guest at boot), snapshot creation and restore for stateful scale-to-zero, and the Multi-Tier Snapshot System (MTSS) that manages snapshot promotion across tiers based on disk usage thresholds and snapshot age.

3.4 Proxy

Inbound traffic flows through a two-layer proxy architecture before reaching microVMs. OpenResty (`ukp-openresty`) handles TLS termination, HTTP routing, and header injection. The C proxy (built as a shared library loaded by OpenResty) handles TCP-level load balancing with per-VM connection tracking, least-connection routing, and scale-to-zero wake-up triggering. The proxy also supports JIT (just-in-time) service creation, where the first inbound request to an unknown domain can trigger automatic service and instance creation without a prior API call.

3.5 CoreDNS

`coreDNS` (`ukp-coredns`) is a fork of upstream `coreDNS` with a custom `ukpdns` plugin that bridges standard DNS queries to `ukpd`'s internal DNS daemon (`iDNS`). It handles VM name resolution for internal traffic and forwards external queries to upstream resolvers. The `eDNS` blacklist — which filters outbound DNS queries by domain pattern — runs inside `ukpd` separately from `coreDNS`.

4 Installation

This chapter covers everything required to deploy Unikraft on a host, from hardware and software prerequisites through package installation to leveraging AMLs. The platform is distributed as a set of Debian packages that install the platform controller, agent, proxy, coreDNS, and networking components. A single install script handles the full setup, including systemd service configuration and initial database creation.

4.1 Requirements

Supported Distributions

The platform currently supports:

- **Debian 13 (Trixie)** — Primary supported platform
- **Debian 12 (Bookworm)** — Supported for distupgrade paths from Debian 11 to 13
- **Debian 11 (Bullseye)** — Deprecated, upgrade to Debian 13 recommended

The installer script (`install.unikraft.cloud`) targets Debian-based systems and uses APT for package management.

Hardware Requirements

Resource	Minimum	Recommended	Notes
CPU	x86_64 with AMD-V or VT-x	Modern multi-core, 3 GHz base clock	Firecracker requires hardware virtualization support. The platform sets the CPU governor to <code>performance</code> . Higher clock frequency improves boot/resume performance.
RAM	Contact Unikraft for sizing guidance	Contact Unikraft for sizing guidance	Must accommodate host services plus VM memory. The <code>--vmm-user-max-memory-mb</code> flag controls the per-user memory budget.
Storage	1 SSD	Multiple PCIe NVMe (U.2, U.3)	Volumes, images, and snapshots are stored on disk. MTSS snapshot lifecycle depends on disk capacity. Prefer directly attached PCIe NVMe drives (avoid RAID controllers), consider RAID-0 or RAID-10.
Network	1 NIC	Contact Unikraft for sizing guidance	The platform binds to a single network interface (<code>NET_IFACE</code>).

Kernel Requirements

The host kernel must support:

- KVM (/dev/kvm) — Required by Firecracker
- TAP devices — For VM networking
- nftables and iptables — For firewall rules (libmn10, libnftnl11)
- cgroups v2 — For resource delegation (systemd Delegate=yes)

System Configuration

The platform sets the following system limits:

File Descriptors

The ukp-platform package installs /etc/security/limits.d/ukp.conf:

```
root    hard    nofile   1048576
root    soft    nofile   1048576
```

This sets the maximum open files to 1M for root, which is necessary because each VM requires multiple file descriptors (TAP device, Firecracker API socket, block devices, etc.).

systemd Configuration

The ukp-platform.service is configured with: - KillMode=process — Only kill ukpd on stop, not guest VMs - Delegate=yes — Allow ukpd to manage its child processes' cgroups - TasksMax=infinity — No task limit

Network Requirements

Requirement	Description
Public IP	The metro node needs a routable IP address on NET_IFACE
DNS	A DNS zone for the API (DNS_ZONE_API, default: unikraft.cloud) and apps (DNS_ZONE_APP, default: unikraft.app)
TLS Certificates	Cloudflare API token with read/write access to both DNS zones (for automated DNS record management)
Internal network	A private IP range for VMs (NET_SEGMENT: 10.0.0.0/8, 172.16.0.0/12 (default), or 192.168.0.0/16); no other network interface may use this range

Software Dependencies

The ukp-platform package depends on:

- systemd
- ukp-config

- ukp-networking
- ukp-firecracker
- ukp-openresty
- ukp-coredns
- ukp-agent
- libukp-rtti (version-matched)
- libpam-modules
- python3 (for ukp-db inspection tool)
- libmnl0 (nftables)
- libnftnl11 (nftables)

Recommended packages:

- systemd-coredump
- cpufrequtils

Storage Layout

The platform stores data across several directories, all under `/var/lib/ukp` by default:

Path (default)	Configuration	Purpose	Filesystem
<code>/var/lib/ukp/data</code>	UKP_DATA	Platform database and state files	Any
<code>/var/lib/ukp/data</code>	UKP_USERDB	User database (JSON file)	Any
<code>/var/lib/ukp/backups</code>	UKP_BACKUPS	UKP Database backups	Any
<code>/var/lib/ukp/volumes</code>	UKP_VOLUMES	Persistent volumes for MicroVMs	XFS with quotas
<code>/var/lib/ukp/snapshots</code>	UKP_SNAPSHOTS	MicroVM snapshots for scale-to-zero	XFS on RAID0/RAID10 recommended
<code>/var/lib/ukp/images</code>	UKP_IMAGES	MicroVM images (kernels, roms, initrds, ...)	XFS on RAID0/RAID10 recommended
<code>/var/run/ukp</code>	UKP_RUNTIME	Runtime sockets and ephemeral state (tmpfs recommended)	tmpfs
<code>/var/log/ukp</code>	UKP_LOGDIR	Platform logs (besides systemd logs)	Any

The platform uses XFS quotas and reflinks for copy-on-write volume cloning, so UKP_VOLUMES requires XFS. Other paths have no specific filesystem requirement, but a single XFS filesystem for all of `/var/lib/ukp` simplifies management. For production deployments, directly attached PCIe NVMe drives (U.2 or U.3) provide the best performance. Configure them in RAID-0 or RAID-10 for maximum throughput, in particular for snapshot operations. Avoid hardware RAID controllers — their controller and upstream PCIe bandwidth limits can bottleneck performance. The majority of the platform's disk I/O is concentrated on the UKP_SNAPSHOTS and UKP_IMAGES directories, so these should be prioritized for high-performance storage.

Example: RAID-10 Setup with 4 drives

Run the following commands as root:

```
# Create RAID-10 array (example with 4 drives, /dev/md0)
mdadm --create /dev/md0 --level=10 --raid-devices=4 \
    /dev/nvme0n1 /dev/nvme1n1 /dev/nvme2n1 /dev/nvme3n1

# Append the new RAID array to mdadm configuration
# Check for duplicates after appending, as mdadm includes all found md arrays
mdadm --detail --scan | tee -a /etc/mdadm/mdadm.conf

# Update the initramfs to include the new RAID array
update-initramfs -k all -u

# Format as XFS
mkfs.xfs /dev/md0

# Mount at /var/lib/ukp
mkdir -p /var/lib/ukp

# Add to fstab for persistence
echo '/dev/md0 /var/lib/ukp xfs defaults,uquota,gquota,pquota,discard 0 2' >> /etc/fstab
systemctl daemon-reload

# Mount the new filesystem
mount /var/lib/ukp
```

4.2 For cloud deployments, use cloud-init to automate this setup. See the [Cloud Images](#) section for examples.

4.3 Packages

The Unikraft platform is distributed as a set of Debian packages installed from the `pkg.unikraft.com` APT repository.

Package Overview

Package	Component	Description
ukp-platform	ukpd	Core platform daemon. Manages VM lifecycle, networking, storage, autoscale, and the platform API.
ukp-agent	Agent	Go sidecar daemon. Handles OCI image pulling, TLS cert provisioning, Prometheus metrics, API log proxying.
ukp-firecracker	Firecracker	MicroVM hypervisor. Launches and manages individual VMs.
ukp-openresty	OpenResty	NGINX + LuaJIT reverse proxy. Handles TLS termination and HTTP routing.
ukp-coredns	coreDNS	DNS server for VM name resolution.
ukp-config	Configuration	Manages <code>/etc/ukp.conf</code> via <code>debconf</code> .
ukp-networking	Networking	TAP device setup, routing rules, and network configuration.
ukp-lego	ACME client	Let's Encrypt client for TLS certificate provisioning.
libukp-rtti	RTTI library	Runtime type information library for the <code>ukp-db</code> inspection tool.

Package Channels

Channel	Description
stable	Production-ready releases
preview	Pre-release versions. May contain features not yet in stable.
staging	Internal testing channel (not publicly available, for access contact Unikraft)

The preview channel is intended for testing and may contain unstable features. Use the stable channel for production deployments. Channels to install can be specified during installation via the `INSTALL_CHANNEL` environment variable.

File Layout

After installation, platform files are located at:

Path	Contents
<code>/usr/lib/ukp/platform/bin/ukpd</code>	ukpd binary
<code>/usr/lib/ukp/platform/tools/</code>	ukp-vol, ukp-db, ukp-db-ix tools

Path	Contents
/usr/lib/ukp/platform/launcher/ukpd	Launcher script
/usr/lib/ukp/platform/config/fs/	Filesystem configuration templates
/usr/lib/ukp/platform/openresty/	OpenResty config, Lua scripts, static files
/usr/lib/ukp/platform/edns-blacklist.txt	eDNS domain blacklist
/usr/lib/ukp/platform/initial_users.json	Default user database template
/usr/lib/ukp/agent/	Agent binary and launcher
/etc/ukp.conf	Platform configuration (generated by ukp-config)
/etc/security/limits.d/ukp.conf	File descriptor limits
/lib/systemd/system/ukp-platform.service	systemd service file

4.4 Installation

Via Install Script (Recommended)

For the install process, an installation token is required. Contact us for more information. Our remote installer is only accessible through this token and during the installation process, it is used for accessing package repositories and the image registry.

The following variables can be set and are recognized by the installation process and can be set like so in order to facilitate an unattended install:

```
curl -sSfL https://install.unikraft.cloud?token=${UKM_TOKEN} | sh
```

The `${UKM_TOKEN}` is provided during enterprise onboarding. The script authenticates with the package repository, configures APT, and installs all packages.

Installation variables can be passed as environment variables:

```
curl -sSfL https://install.unikraft.cloud?token=${UKM_TOKEN} | \
  DNS_HOSTNAME=fra1 \
  NET_IFACE=eth0 \
  PROMETHEUS_API_TOKEN=<token> \
  CLOUDFLARE_EMAIL=admin@example.com \
  CLOUDFLARE_TOKEN=<cf_token> \
  sh
```

Variable	Default	Purpose
CLOUDFLARE_EMAIL ¹		Used during the CloudFlare DNS challenge for dynamically provisioned subdomains.
CLOUDFLARE_TOKEN ²		Access token used to dynamically set DNS entries.
DNS_HOSTNAME	\$(uname -n)	Name of the metro without domain/zone, e.g. fra0 or nyc1.
DNS_ZONE_API	unikraft.cloud	Top-level DNS domain for API.
DNS_ZONE_APP	unikraft.app	Top-level DNS domain for instances.
OTEL_COLLECTOR_ADDR		Sets the address of the remote OpenTelemetry instance to send logs.
OTEL_COLLECTOR_TOKEN		Authentication token to access remote OpenTelemetry instance.
PROMETHEUS_API_TOKEN		Sets the bearer token used to authenticate against all exposed metrics endpoint.
NET_IFACE		Name of public network interface. Auto-detected if not specified.
NET_SEGMENT	172.16.0.0/12	Internal network segment to be used for guest instances.
NET_COUNT	256	Number of tenant networks to create.
UKPD_ARGS		Extra arguments for the Cloud Platform Daemon.
CUSTOM_MPOINTS		List of mount points for Cloud Platform to wait before starting up (space separated).
INSTALL_CHANNEL	stable	Specifies the installation channel to use. Options are <code>stable</code> and <code>preview</code> . For access to staging contact Unikraft team.
NO_CONFIG	no	Skip configuration steps, only perform package installation. Useful for repairing or troubleshooting.
NO_INSTALL	no	Skip package installation and configuration, only perform APT setup and install Unikraft signing keys. Useful for repairing or troubleshooting.

Via APT (Manual)

Using the installer script is the recommended installation method, but you can also set up the APT repository and install packages manually. Please note that the installer script performs additional setup steps beyond just installing packages, such as configuring CPU governor settings.

¹The supplied CloudFlare token (CLOUDFLARE_TOKEN) requires read/write permissions on both DNS_ZONE_API and DNS_ZONE_APP zones.

²The supplied CloudFlare token (CLOUDFLARE_TOKEN) requires read/write permissions on both DNS_ZONE_API and DNS_ZONE_APP zones.

To add the cloud repository to your Debian installation, proceed with the following steps:

1. In the following, `${PKG_TOKEN}` refers to your generated token for the installation (not username, not password) which you will receive during your enterprise onboarding.
2. Download and install Unikraft keyring:

```
# Download and add the GPG key
sudo curl -u "api:${PKG_TOKEN}" -fsSL \
  -o /usr/share/keyrings/unikraft-cloud.gpg \
  https://pkg.unikraft.com/debian/cloud/keys/cloud.gpg
```

3. Set up `/etc/apt/auth.conf.d/unikraft.conf`:

```
cat <<EOF | sudo tee /etc/apt/auth.conf.d/unikraft.conf > /dev/null
machine pkg.unikraft.com
login api
password ${PKG_TOKEN}
EOF

sudo chmod -c 0600 /etc/apt/auth.conf.d/unikraft.conf
```

4. Add cloud to APT sources:

```
cat <<EOF | sudo tee /etc/apt/sources.list.d/unikraft-cloud.sources > /dev/null
Types: deb
URIs: https://pkg.unikraft.com/debian/cloud/
Suites: $(. /etc/os-release && echo "${VERSION_CODENAME}")
Components: stable
Signed-By: /usr/share/keyrings/unikraft-cloud.gpg
EOF
```

Valid components are `stable` and `preview`. `preview` contains feature previews and should only be installed on non-production nodes.

5. Install the relevant components:

```
apt update
apt install -y \
  unikraft-keyring \
  ukp-agent \
  ukp-config \
  ukp-coredns \
  ukp-firecracker \
  ukp-lego \
  ukp-networking \
  ukp-openresty \
  ukp-platform
```

6. Start the platform:

```
systemctl start ukp-online.target
```

4.5 Cloud Images

We provide pre-built cloud images for AWS, GCP, and others that are ready to launch with the platform. Please contact us directly to receive provisioned cloud images for your account. For AWS, we require your AWS Account ID or an Organizational Unit (OU) ARN. GCP images are shared directly with your GCP project.

Cloud-Init Configuration

When launching a cloud instance, provide a cloud-init userdata script to configure the platform. A typical cloud-init configuration contains:

```
#cloud-config
hostname: xyz0
preserve_hostname: false

chpasswd:
  expire: false
  users:
    - name: admin
      password: changeme
      type: text

write_files:
  - path: /var/lib/ukp/data/users.json
    owner: 'root:root'
    permissions: '0644'
    content: |
      [
        {
          "uuid": "<UUID>",
          "name": "<ORG_NAME>",
          "network_id": 1,
          "auth_token": [
            "<UKC_TOKEN>"
          ],
          "permissions": ["developer"]
        }
      ]
  - path: /etc/ukp.d/lego/envfile
    owner: 'root:root'
    permissions: '0600'
```

```
content: |
    LEGO_DNS_PROVIDER=route53
    LEGO_CERT_EMAIL=sysadmin+xyz0@example.com
    AWS_ACCESS_KEY_ID=your_key_id
    AWS_SECRET_ACCESS_KEY=your_secret_access_key
    AWS_REGION=aws-region
    AWS_HOSTED_ZONE_ID=your_hosted_zone_id

- path: /etc/ukp.conf
  owner: 'root:root'
  permissions: '0600'
  content: |
    ## =====
    ## Global configuration
    ## =====
    UKP_DATA="/var/lib/ukp/data"
    UKP_BACKUPS="/var/lib/ukp/backups"
    UKP_VOLUMES="/var/lib/ukp/volumes"
    UKP_SNAPSHOTS="/var/lib/ukp/snapshots"
    UKP_IMAGES="/var/lib/ukp/images"
    UKP_RUNTIME="/var/run/ukp"
    UKP_USERDB="/var/lib/ukp/data/users.json"
    UKP_LOGDIR="/var/log/ukp"
    UKP_PROMETHEUS_API_TOKEN=""
    UKP_CUSTOM_MPOINTS=("/var/lib/ukp")

    ## =====
    ## Platform daemon
    ## =====
    UKPD_EXTRA_ARGS=()
    UKPD_EXTRA_ARGS+=("--vmm-initrd-map-shared")
    UKPD_EXTRA_ARGS+=("--vm-max-instances=10000")
    UKPD_EXTRA_ARGS+=("--stor-max-volumes=5000")
    UKPD_EXTRA_ARGS+=("--vmm-mtss-enable=1,1,0")
    UKPD_EXTRA_ARGS+=("--vmm-mtss-min-age-s=600,0")

    ## =====
    ## Network/IP configuration
    ## =====
    NET_MODE="isolates"
    NET_COUNT="1000"

    # NET_SEGMENT can be either:
    # - 10.0.0.0/8
    # - 172.16.0.1/12
    NET_SEGMENT="10.0.0.0/8"

    ## =====
    ## Public DNS
```

```
## =====
DNS_ZONE_API="mongodb-dev.com"
DNS_ZONE_APP="mongodb-dev.app"

## =====
## Internal DNS
## =====
IDNS_HOSTNAME="ns"
IDNS_ZONE="internal"
IDNS_EMAIL="ns@internal"

## =====
## Platform agent
## =====
AGENT_PULL_NATS_USER="<ORG_NAME>"
AGENT_PULL_NATS_TOKEN="<NATS_TOKEN>"
AGENT_PULL_HARBOR_TOKEN="<UKC_TOKEN>"
AGENT_PULL_INCLUDE="<ORG_NAME>"

## =====
## Lego daemon
## =====
LEGO_ENVFILE="/etc/ukp.d/lego/envfile"
```

The `bootcmd` section supports further customization. The platform does not start before all `bootcmd` commands and the `write_files` module complete. Because `runcmd` runs while the platform is already initializing, some commands may not finish before startup. The images use all available disk space on the boot drive for UKP data (`/var/lib/ukp` using XFS).

Unikraft also provides **nodata** image variants that do not include a pre-configured volume for UKP data. These images suit environments where separate storage drives (e.g., NVMe) provide better performance. The operator sets up cloud-init to configure the additional drives and mounts them to `/var/lib/ukp` before the platform starts. The following cloud-init snippet (extending the previous example) scans for additional NVMe drives and configures a RAID 0 array with XFS for UKP data (`/var/lib/ukp`). **nodata** images do not automatically start the platform, so the `runcmd` section includes commands to start the platform after the RAID array is configured.

```
bootcmd:
# Create the RAID 0 array with all unused NVMe devices (no partitions).
# bootcmd runs before the write_files module, so we can even populate
# files in the RAID array
- |
DEVS=$(lsblk -dno NAME | grep -E '^nvme[0-9]+n[0-9]+$' | while read D; do
    [ $(lsblk -no NAME /dev/$D | wc -l) -eq 1 ] && echo /dev/$D
done)
[ -n "$DEVS" ] && /usr/local/sbin/configure-ukp-data-raid0 $DEVS

runcmd:
```

```
# Start the platform
- systemctl enable ukp-online.target
- systemctl start ukp-online.target
```

4.6 Client-side Tooling

You can quickly and easily install unikraft CLI tool, the companion client-side by using the interactive installer. Run the following command to install the program locally:

```
curl --proto '=https' --tlsv1.2 -fsSL https://unikraft.com/cli/install.sh | sh
```

This installs into the first preferred bin directory found in your \$PATH (falling back to \$HOME/.local/bin if none are available). Use environment variable UNIKRAFT_CLI_INSTALL_BIN_DIR to customize the installation directory.

The source code for the CLI tool is available at <https://github.com/unikraft/cli.git>.

macOS

```
brew install unikraft/cli/unikraft
```

Debian/Ubuntu

```
# Update and install dependencies
sudo apt update
sudo apt install ca-certificates curl

# Download and add the GPG key
sudo curl -fsSL \
  -o /usr/share/keyrings/unikraft-cli.gpg \
  https://pkg.unikraft.com/debian/cli-apt/keys/cli-apt.gpg

sudo tee /etc/apt/sources.list.d/unikraft-cli.sources <<EOF
Types: deb
URIs: https://pkg.unikraft.com/debian/cli-apt/
Suites: $(. /etc/os-release && echo "${VERSION_CODENAME}")
Components: stable
Signed-By: /usr/share/keyrings/unikraft-cli.gpg
EOF

# Update and install
sudo apt-get update
```

```
sudo apt-get install unikraft-cli unikraft-cli-keyring
```

Fedora/RHEL/Rocky/Alma

```
# Add the Unikraft CLI repository
sudo tee /etc/yum.repos.d/unikraft-cli-rpm.repo <<EOF
[unikraft-cli]
name=unikraft-cli
baseurl=https://pkg.unikraft.com/rpm/cli-rpm/
gpgcheck=0
enabled=1
EOF

# Install the CLI at the specific pre-release version
yum install unikraft-cli
```

See [alternative installation instructions](#) for other platforms.

4.7 CLI Configuration

```
# Linux: ~/.config/unikraft/config.yaml
# MacOS: ~/Library/Application\ Support/unikraft/config.yaml
profile: default
profiles:
  default:
    type: cloud
    organization: <org-name>
    token: <api-token>
    metros:
      - name: <metro-name> # e.g. fra
        endpoint: <metro-endpoint> # e.g. https://api.fra.unikraft.cloud
        country: <metro-country> # e.g. de
        insecure: false # skip tls verification (avoid for production use)
```


5 Platform Startup

The platform uses systemd to manage service startup, with a chain of targets and services that ensure components start in the correct order. Understanding the startup sequence is essential for troubleshooting boot issues and planning maintenance operations.

5.1 Systemd Targets

The platform defines four systemd targets that organize services into logical startup phases:

Target	Description
ukp-config.target	Configuration phase. Reached when <code>/etc/ukp.conf</code> is available and cloud-init (if present) has completed.
ukp-pre.target	Pre-platform phase. Reached when filesystem barriers, firewall, and networking prerequisites are ready.
ukp.target	Platform phase. Reached when all core platform services (ukpd, agent, proxy, DNS, networking) are running.
ukp-online.target	Online phase. Reached when the platform is fully operational and ready to serve requests.

Operators should use `ukp-online.target` as the primary control point:

```
# Check if platform is fully online
systemctl list-dependencies ukp-online.target

# Start the platform
systemctl start ukp-online.target

# Stop the platform (but keep firewall)
systemctl stop ukp-pre.target

# Stop the entire platform (including firewall and filesystem barriers)
systemctl stop ukp-config.target
```

5.2 Startup Order

The targets form a linear dependency chain:

```
cloud-config.target    (if present, covers: bootcmd, write_files)
|
v
ukp-config.target      (Configuration available)
|
```

```
+ ukp-fs-barrier.service
+ ukp-firewall.service
V
ukp-pre.target      (Prerequisites ready: firewall, mount points)
|
+ ukp-vnet.service
+ ukp-platform.service
+ ukp-platform-postdelay.service
+ ukp-agent.service
+ ukp-openresty.service
+ ukp-coredns.service
V
ukp.target          (Core services running)
|
V
ukp-online.target   (Platform fully operational)
```

5.3 Services

ukp-fs-barrier.service

Package: ukp-config

Waits for custom mount points defined in `UKP_CUSTOM_MPOINTS` to become available before allowing dependent services to start. This ensures that platform data directories stored on external volumes, network-attached storage, or encrypted partitions are mounted before services attempt to access them.

Property	Value
Type	oneshot
Requires	ukp-config.target, local-fs.target, remote-fs.target
After	ukp-config.target, local-fs.target, remote-fs.target
Before	ukp-pre.target

The service checks mount points every 15 seconds and blocks indefinitely until all specified paths are available. If `UKP_CUSTOM_MPOINTS` is empty, the service completes immediately.

See the [Configuration](#) documentation for details on configuring custom mount points.

ukp-firewall.service

Package ukp-networking

Configures host firewall rules using iptables, ip6tables, and ipsets. Creates network isolation rules, NAT for guest traffic, and prepares ipsets for VM network registration.

Property	Value
Type	oneshot
Requires	ukp-config.target
After	ukp-config.target, network-online.target
Before	ukp-pre.target

See the [Networking](#) documentation for details on firewall rules and customization hooks.

ukp-vnet.service

Package: ukp-networking

Creates and configures TAP devices for VM networking. Each TAP device is assigned an IP address from the configured NET_SEGMENT and registered with the firewall ipsets.

Property	Value
Type	oneshot
Requires	ukp-pre.target
After	ukp-pre.target, ukp-firewall.target
Before	ukp.target

See the [Networking](#) documentation for details on TAP device configuration.

ukp-platform.service

Package ukp-platform

The core platform controller (ukpd). Manages VM lifecycle, storage, networking assignment, autoscaling, and the platform API.

Property	Value
Type	simple
Requires	ukp-pre.target, ukp-vnet.service
After	ukp-pre.target, ukp-vnet.service
Before	ukp.target
KillMode	process
Delegate	yes
LogNamespace	ukp-platform

The service uses `KillMode=process` to ensure that stopping `ukpd` does not terminate guest VMs. The `Delegate=yes` setting allows `ukpd` to manage cgroups for child processes. Logs are written to a separate journal namespace (`ukp-platform`) and can be viewed with:

```
journalctl --namespace=ukp-platform -u ukp-platform.service
```

ukp-platform-postdelay.service

Package: `ukp-platform`

A service that introduces a 10-second delay after `ukpd` starts, ensuring that `ukpd`'s API socket is ready before dependent services (like OpenResty and coreDNS) attempt to connect.

Property	Value
Type	oneshot
Requires	ukp-platform.service
After	ukp-platform.service
Before	ukp.target

ukp-agent.service

Package: `ukp-agent`

The platform agent handles auxiliary tasks: OCI image management, TLS certificate provisioning via ACME, Prometheus metrics export, and system health checks.

Property	Value
Type	simple
Requires	ukp-platform.service
After	ukp-platform.service
Before	ukp.target

The agent communicates with `ukpd` over a Unix domain socket using the HAC (Host-Agent-Communication) protocol.

ukp-openresty.service

Package: `ukp-openresty`

The reverse proxy (OpenResty/NGINX + LuaJIT) handles TLS termination, HTTP routing to VMs, and header injection for request tracing.

Property	Value
Type	simple
Requires	ukp-pre.target, ukp-vnet.service
After	ukp-pre.target, ukp-vnet.service
Before	ukp.target
Restart	on-failure
LogNamespace	ukp-platform

OpenResty reloads automatically when TLS certificates are renewed by the lego renewal hook. Logs are written to the ukp-platform journal namespace.

ukp-coredns.service

Package: ukp-coredns

The DNS server for VM name resolution. Includes a custom ukpdns plugin that bridges DNS queries to ukpd's internal DNS daemon (iDNS).

Property	Value
Type	simple
Requires	ukp-pre.target, ukp-vnet.service
Wants	ukp-platform.service, ukp-platform-postdelay.service
After	ukp-pre.target, ukp-platform.service, ukp-platform-postdelay.service, ukp-vnet.service
Before	ukp.target
Restart	on-failure
LogNamespace	ukp-platform

The service waits for ukpd to be ready (via ukp-platform-postdelay.service) before starting. Logs are written to the ukp-platform journal namespace.

5.4 Startup Troubleshooting

Check Target Status

```
# Check if all targets are reached
systemctl is-active ukp-config.target ukp-pre.target ukp.target ukp-online.target

# List failed units
```

```
systemctl list-units --state=failed 'ukp*'
```

View Startup Order

```
# Show dependency tree
systemctl list-dependencies ukp-online.target

# Show reverse dependencies (what depends on a service)
systemctl list-dependencies --reverse ukp-platform.service
```

Debug Startup Issues

```
# Check service status
systemctl status ukp-platform.service

# View service logs
journalctl -u ukp-platform.service --since boot

# Check why a target is not reached
systemctl list-dependencies ukp-pre.target --all | grep -v '□'
```

Common Startup Failures

Symptom	Likely Cause	Resolution
ukp-fs-barrier.service stuck	Custom mount point not available	Check UKP_CUSTOM_MPOINTS in /etc/ukp.conf and verify mount points. Check logs with <code>journalctl -u ukp-fs-barrier.service</code>
ukp-firewall.service failed	Network interface not ready	Verify NET_IFACE in /etc/ukp.conf or wait for network-online.target. Check logs with <code>journalctl -u ukp-firewall.service</code>
ukp-platform.service failed	Missing configuration or database corruption	Check logs with <code>journalctl -u ukp-platform.service</code> and <code>/var/log/ukp/platform/ukpd.log</code>
ukp-coredns.service failed	Port 53 conflict	Ensure no other DNS server listens on port 53 (including systemd-resolved's DNSStubListener). Check logs with <code>journalctl -u ukp-coredns.service</code> .

6 Platform Configuration

The platform is configured through `/etc/ukp.conf`, a bash-syntax configuration file that defines paths, networking parameters, DNS settings, and service credentials. This file is read by all platform components — `ukpd`, the agent, networking services, and `coreDNS` — to ensure consistent configuration across the system.

6.1 Managing Configuration

The configuration file is generated and managed by the `ukp-config` package through `debconf`:

```
# Reconfigure interactively
dpkg-reconfigure ukp-config

# View current configuration
cat /etc/ukp.conf
```

Each configuration option is defined by each `ukp` package. The services of these packages read the configuration file at startup and apply the settings accordingly. They also implement backwards compatibility, so older versions of the configuration file will still work with newer versions of the platform.

After modifying `/etc/ukp.conf`, reload (some need restarting) the affected services for changes to take effect.

6.2 Configuration Sections

Global Configuration

These variables define the filesystem paths where the platform stores data, and global settings that apply across all components.

Variable	Default	Description
UKP_DATA	<code>/var/lib/ukp/data</code>	Root directory for platform databases (VMDB, IFACEDB, VOLDB, user database).
UKP_BACKUPS	<code>/var/lib/ukp/backups</code>	Directory for platform backups.
UKP_VOLUMES	<code>/var/lib/ukp/volumes</code>	Directory for persistent volume storage.
UKP_SNAPSHOTS	<code>/var/lib/ukp/snapshots</code>	Directory for MTSS snapshot storage.
UKP_IMAGES	<code>/var/lib/ukp/images</code>	Directory for OCI image storage.
UKP_RUNTIME	<code>/var/run/ukp</code>	Runtime directory for sockets, PID files, and ephemeral state.
UKP_USERDB	<code>/var/lib/ukp/data/users.json</code>	Path to the user database file.
UKP_LOGDIR	<code>/var/log/ukp</code>	Directory for platform log files.

Variable	Default	Description
UKP_PROMETHEUS_API_TOKEN	<i>(empty)</i>	Bearer token for authenticating Prometheus scrape requests to the public metrics endpoint.

Custom Mount Points

The `UKP_CUSTOM_MPOINTS` variable specifies mount points that must be available before platform services start. This is useful when platform data is stored on volumes that require manual mounting (e.g., network-attached storage, encrypted volumes, or separate partitions).

```
# Wait for /var/lib/ukp to be mounted before starting services
UKP_CUSTOM_MPOINTS=()
UKP_CUSTOM_MPOINTS+=("/var/lib/ukp")
UKP_CUSTOM_MPOINTS+=("/var/lib/ukp/volumes")
```

The `ukp-fs-barrier.service` checks that all specified mount points are available and blocks service startup until they are ready. This ensures that the platform does not start with missing or inaccessible storage, preventing potential data loss or service failures. The intention is that the operator SSH into the host and mount the required volumes to allow the platform to proceed with startup.

Platform Daemon

Configuration specific to `ukpd`, the core platform controller.

Variable	Default	Description
UKPD_EXTRA_ARGS	<i>(empty array)</i>	Additional command-line arguments passed to <code>ukpd</code> at startup.

Use `UKPD_EXTRA_ARGS` to pass flags that override `ukpd` defaults without modifying the `systemd` unit file:

```
UKPD_EXTRA_ARGS=()
UKPD_EXTRA_ARGS+=("--vmm-user-max-memory-mb" "8192")
UKPD_EXTRA_ARGS+=("--api-listen" "0.0.0.0:443")
```

See the [Platform Controller](#) documentation for available `ukpd` flags.

Network Configuration

These variables configure the host network interface and the private network segment used for VM networking.

Variable	Default	Description
NET_IFACE	<i>(auto-detected)</i>	The host network interface for public traffic. If empty, the interface with the default route is used.
NET_MODE	isolates	Network isolation mode. Only isolates is supported.
NET_SEGMENT	172.16.0.0/12	Private network segment for VM IP addresses. Must be 10.0.0.0/8, 172.16.0.0/12, or 192.168.0.0/16.
NET_COUNT	256	Number of TAP devices to pre-provision. Each device supports one VM.

Note: Earlier platform versions supported a `subnets` mode for `NET_MODE`, which has been removed.

The network segment determines the pool of IP addresses available for VMs. In `isolates` mode, each VM gets its own /30 network carved from the segment. Ensure the segment does not overlap with addresses already in use on the host or reachable networks.

See the [Networking](#) documentation for details on TAP device provisioning and firewall configuration.

Public DNS

These variables configure the DNS zones used for API endpoints and application instances.

Variable	Default	Description
DNS_HOSTNAME	<i>(hostname -s)</i>	The metro node hostname used in DNS records. If empty, detected from <code>hostname -s</code> .
DNS_HOSTNAME_NOAUTO	n	Set to y to disable automatic hostname detection and use an empty hostname.
DNS_ZONE_API	<i>(hostname -d)</i>	DNS zone for API endpoints. APIs resolve as <code><endpoint>.<DNS_HOSTNAME>.<DNS_ZONE_API></code> .
DNS_ZONE_APP	<i>(hostname -d)</i>	DNS zone for application instances. Instances resolve as <code><name>.<DNS_HOSTNAME>.<DNS_ZONE_APP></code> .

Example DNS resolution with `DNS_HOSTNAME=fra0`, `DNS_ZONE_API=unikraft.cloud`, and `DNS_ZONE_APP=unikraft.app`:

Endpoint	Resolves to
API	api.fra0.unikraft.cloud
Index	index.fra0.unikraft.cloud
Instance myapp	myapp.fra0.unikraft.app

Endpoint	Resolves to
----------	-------------

Internal DNS

These variables configure the internal DNS zone used for VM-to-VM name resolution.

Variable	Default	Description
IDNS_HOSTNAME	ns	Nameserver hostname for the internal DNS zone.
IDNS_ZONE	internal	The internal DNS zone name. VMs resolve each other as <hostname>.<IDNS_ZONE>.
IDNS_EMAIL	ns@internal	Email address for the SOA record.

Internal DNS allows VMs belonging to the same user to resolve each other by name. For example, with `IDNS_ZONE=internal`, a VM named `backend` resolves as `backend.internal`.

See the [Networking](#) documentation for details on iDNS and coreDNS configuration.

Platform Agent

These variables configure the platform agent, which handles OCI image pulling, TLS certificate provisioning, and metrics collection.

Variable	Default	Description
AGENT_PULL_NATS_USER	<i>(empty)</i>	NATS username for image pull notifications.
AGENT_PULL_NATS_TOKEN	<i>(empty)</i>	NATS authentication token.
AGENT_PULL_HARBOR_TOKEN	<i>(empty)</i>	Base64-encoded credentials for Harbor registry access.
AGENT_PULL_INCLUDE	<i>(empty)</i>	Comma-separated list of image tags to include in pulls (e.g., <code>official</code>).
AGENT_PULL_IGNORE	<i>(empty)</i>	Comma-separated list of image tags to ignore (e.g., <code>test</code>).
AGENT_OTEL_ADDR	<i>(empty)</i>	OpenTelemetry collector endpoint for distributed tracing.
AGENT_OTEL_TOKEN	<i>(empty)</i>	Authentication token for the OpenTelemetry collector.

See the [Agent](#) documentation for details on image pulling and certificate management.

TLS Certificate Provisioning

These variables configure automatic TLS certificate provisioning via ACME (Let's Encrypt) using Cloudflare DNS validation.

Variable	Default	Description
CLOUDFLARE_DNS_API_TOKEN	<i>(empty)</i>	Cloudflare API token with DNS zone edit permissions. Required for DNS-01 challenge.
CLOUDFLARE_DNS_EMAIL	<i>(empty)</i>	Email address associated with the Cloudflare account.
LEGO_ENVFILE	<i>(empty)</i>	Path to the environment file used by the <code>lego</code> ACME client instead of Cloudflare credentials. This enables more options for different ACME providers.

The platform uses the `lego` ACME client to automatically provision and renew TLS certificates. The Cloudflare API token must have `Zone:DNS:Edit` permissions for the zones specified in `DNS_ZONE_API` and `DNS_ZONE_APP`.

See the [Certificates](#) documentation for details on certificate management and the `lego` client.

Services Forwarding (deprecated)

These variables configure optional port forwarding for PostgreSQL databases and SMB/CIFS file sharing services.

Variable	Default	Description
REMOTE_PSQLE	<i>(empty)</i>	IP address of a remote PostgreSQL server. Traffic to port 5432 is forwarded to this address.
LOCAL_PSQLE	<i>(empty)</i>	Set to <code>true</code> to allow VM access to a local PostgreSQL server on port 5432.
REMOTE_SMBD	<i>(empty)</i>	IP address of a remote SMB/CIFS server. Traffic to port 445 is forwarded to this address.
LOCAL_SMBD	<i>(empty)</i>	Set to <code>true</code> to allow VM access to a local SMB/CIFS server on port 445.

Note: Only one of `REMOTE_*` or `LOCAL_*` should be set for each service. Setting `REMOTE_*` takes precedence.

6.3 Applying Changes

After modifying `/etc/ukp.conf`:

1. **Validate syntax** — The file uses bash syntax. Ensure no syntax errors exist.
2. **Restart services** — Most changes require restarting `ukp-platform.service`:

```
update-ukp-certificates      # if ACME/TLS settings changed
systemctl reload ukp-openresty # if ACME/TLS settings, DNS_HOSTNAME, or DNS_ZONE_API/APP
                               ⇨ changed
systemctl reload ukp-firewall # on NET_IFACE change or firewall rule changes (ipsets not
                               ⇨ updated!)
systemctl reload ukp-vnet     # on NET_COUNT increase
systemctl restart ukp-platform # for most other changes
systemctl restart ukp-agent    # if agent-related settings changed
```

3. **Network changes** — If `NET_IFACE`, `NET_SEGMENT`, or `NET_COUNT` changed, restart networking:

```
systemctl restart ukp-firewall ukp-vnet
```

Note: Restarting (instead of reloading) `ukp-firewall` and `ukp-vnet` causes temporary network disruption for all VMs. Other services also restart. Plan for downtime when applying network configuration changes. Prefer changes that activate with a reload instead. **Warning:** Changing `NET_SEGMENT` or reducing `NET_COUNT` on a running system disrupts active VMs and can cause failures. Plan for maintenance downtime when modifying network configuration. Migrate instances to another host before making changes.

7 Platform Controller

The platform controller, ukpd, is the central process that runs the Unikraft platform on each host. Every operation — starting an instance, allocating a network interface, provisioning a certificate, scaling a service — flows through ukpd. It owns all persistent state, coordinates with the agent and proxy, and exposes the REST API that clients interact with. Understanding the platform controller is essential for operating, configuring, and troubleshooting the platform.

The controller is a multi-threaded C process packaged as `ukp-platform`.

7.1 Initialization

On startup, the controller initializes its subsystems in a fixed order:

1. Parse command-line arguments
2. Initialize logging (file, socket, or journald backend)
3. Open or create the workspace directory
4. Initialize the cluster subsystem (NODEDB)
5. Open databases: IMGDB, IFACEDB, CERTDB, SRVDB, VOLDB, VMDB
6. Initialize networking (TAP interfaces, firewall via nftables)
7. Initialize the monitor subsystem (CPU, health, deadline)
8. Initialize the VMM subsystem responsible for VM execution
9. Initialize the autoscale engine
10. Connect to the agent via Host Agent Client (HAC)
11. Start the API servers (frontend, backend, metrics)
12. Start the message API servers (VMM, proxy)
13. Enter the main event loop

7.2 Subsystems

API Server

The controller exposes three API endpoints:

Endpoint	Default Address	Purpose
Frontend API	127.0.0.1:45232	External client API (instances, services, volumes, certificates, images, autoscale, ...)
Backend API	127.0.0.1:45235	Inter-node cluster API
Metrics API	127.0.0.1:45233	Prometheus scrape endpoint

The frontend API serves the `/v1/` REST endpoints that the `kraft` and `unikraft` CLIs and other clients use. OpenResty reverse-proxies external HTTPS traffic to this endpoint.

Worker Pools

The controller uses multiple worker pools for multiplexed I/O and asynchronous job processing. The pools run jobs like API request processing, snapshot creation, volume operations, and scale-to-zero timers.

Periodic background jobs (e.g., cleaning up zombie instances) execute at least every `interval_ms` (default: 250 ms).

Configuration: - `--wrk-workers` — Number of worker threads in main pool (default: 4) - `--wrk-interval-ms` — Interval in milliseconds for background job execution (default: 250) - `--wrk-zombie-limit` — Maximum zombie object count before active triggering of background cleanup (default: 10) - `--vmm-snapshot-max-workers` — Maximum number of threads in dynamic snapshot worker pool (default: 6)

In addition, to the worker pools the controller employs various dedicated threads, for example, for monitoring.

Platform Component IPC

The controller uses multiple channels to communication with other components on the system. All of the following channels use binary protocols for maximum efficiency.

Direction	Type	Contents
proxy → ukpd	POSIX message queue (ukp-proxy-notify)	Connection events, service resolution requests, autoscale requests
proxy → ukpd	POSIX message queue (ukp-proxy-stats-notify)	Per-VM TTFB stats collection
proxy ↔ ukpd	Anonymous shared memory	Per-service proxy-shared control data
proxy ↔ ukpd	Shared memory-mapped DB (vmdb.shm)	Per-VM proxy-shared control data
VMM → ukpd	POSIX message queue (ukp-vmm-notify)	VM lifecycle events (e.g., boot complete), network action events (e.g., new listen socket)
ukpd → VMM	UNIX domain socket (/var/lib/ukp/platform/<U	VMM control messages
ukpd → VM	VSOCK over UNIX domain socket (/var/lib/ukp/platform/<UUID>/v.sock)	Messages to the VM guest (e.g., scale-to-zero notifications)
ukpd ↔ agent	UNIX domain socket (/var/run/ukp/agent.api)	Image requests, user management, and others

In addition, the controller sends POSIX signals to VMMs for the following operations:

Signal	Operation
SIGUSR1(10)	Initiate orderly VM shutdown
41	Initiate quick VM shutdown
SIGTERM(15)	Force kill
SIGQUIT(3)	Force kill with dump option

The controller itself has handlers for the following signals:

Signal	Behavior
SIGTERM / SIGINT	Graceful shutdown — stops the worker loop and cleans up
SIGHUP	Reopens log files (for log rotation)

7.3 Persistent Databases

The controller manages all state in objects of different types:

- Instances
- Images
- Network interfaces
- Service groups
- Certificates
- Volumes
- Nodes

The objects are stored in binary database files within the workspace directory (default: `/var/lib/ukp/platform/`):

File	Database	Contents
vmdb.bin	VMDB	Instances, instance templates, checkpoints
imgdb.bin	IMGDB	Image metadata
ifacedb.bin	IFACEDB	Network interfaces
srvdb.bin	SRVDB	Service groups incl. domains, and autoscale configurations
certdb.bin	CERTDB	TLS certificates
voldb.bin	VOLDB	Volumes
nodedb.bin	NODEDB	Node metadata in cluster setups

These databases are memory-mapped at runtime. The `ukp-db` and `ukp-db-ix` tools provide command-line inspection, repair, and import/export operations (see [Troubleshooting](#)).

Each entry in the databases has an inline heap for variably-sized data. If the inline heap of an entry is exhausted, a DB-specific global heap is used which resides in the `*.heap` files.

Database Inspection

The `ukp-db` tool provides command-line inspection and repair for these databases. It uses a type introspection library for safe database access.

The `ukp-db-ix` tool supports import/export of VM entries and their dependencies (volumes, services, certificates, interfaces) between hosts via JSON.

ukp-db Usage

```
ukp-db /path/to/workspace           # Open all DBs interactively
ukp-db --db vmdb /path/to/vmdb.bin  # Open specific DB
ukp-db --exec "print(vmdb[0])"      # Execute inline script
ukp-db --exec-file script.py        # Execute script file

# In the interactive shell:
for vm in vmdb: print(vm)           # Iterate entries
vmdb["uuid-here"]                   # UUID lookup
vmdb.filter_active()                # Filter active entries
vmdb.header                         # DB header info
vmdb.rebuild_lists()                # Repair free/zombie lists
```

ukp-db-ix Usage

```
ukp-db-ix export WORKSPACE uuids.txt > out.json  # Export VMs + deps
ukp-db-ix import WORKSPACE in.json              # Import entries
ukp-db-ix import -n WORKSPACE in.json           # Dry run
ukp-db-ix list-uuids in.json vmdb               # List exported UUIDs
ukp-db-ix list-files WORKSPACE in.json snapshots # List files to copy
```

Database Reset

Do a full reset of your installation (i.e., delete all VMs, services, etc.):

1. Stop all `ukp-*` services
2. Terminate all running VMs
3. Delete the workspace directory
4. Delete all volume files and directories in the configured volumes path
5. Delete all images in the configured images path
6. Restart the `ukp-*` services

7.4 User Management

The controller loads user definitions from a JSON file specified by `--users` (default: `users.json` in the workspace). Each user entry defines authentication credentials, resource quotas, and permissions that control what the user can do on the platform.

When `--no-authorization` is set **and** no `--users` file is provided, all API clients receive full administrative access with no authentication required. All resource quotas are effectively bypassed.

This mode is intended for development and testing environments only and must never be used in production.

users.json Format

The user database is a JSON array of user objects:

```
[
  {
    "uuid": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example-user",
    "auth_token": "<token-string-or-array>",
    "permissions": ["developer"],
    "network_id": 1,
    "uid": 100001,
    "disabled": false,
    "vmdb": {
      "max_instances": 100,
      "min_memory_mb": 16,
      "def_memory_mb": 128,
      "max_memory_mb": 16384,
      "min_vcpus": 1,
      "max_vcpus": 8
    },
    "vmm": {
      "max_vcpus": 64,
      "max_memory_mb": 65536
    },
    "net": {
      "max_service_groups": 10,
      "max_services": 100,
      "max_certificates": 20
    },
    "stor": {
      "max_volumes": 50,
      "min_volume_mb": 1,
      "max_volume_mb": 10240,
      "max_total_volume_mb": 102400
    },
    "autoscale": {
      "min_size": 0,
      "max_size": 50
    }
  }
]
```

Field Reference

Field	Type	Required	Description
uuid	string (UUID)	Yes	Unique identifier for the user
name	string	Yes	Human-readable user name
auth_token	string or string[]	No	API authentication token(s). Can be a single string or an array of strings.
permissions	string[]	No	List of permission types (see below)
network_id	integer	Yes	Network segment identifier for the user
uid	integer	No	Unix UID (default: 100000 + network_id)
disabled	boolean	No	Whether the user is disabled (default: false)
vmdb	object	No	Instance creation quotas (max_instances, min_memory_mb, def_memory_mb, max_memory_mb, min_vcpus, max_vcpus)
vmm	object	No	Runtime resource limits (max_vcpus, max_memory_mb)
net	object	No	Networking quotas (max_service_groups, max_services, max_certificates)
stor	object	No	Storage quotas (max_volumes, min_volume_mb, max_volume_mb, max_total_volume_mb)
autoscale	object	No	Autoscale limits (min_size, max_size)

Permission Types

Permission	Description
root	Full access to all platform operations
override_edns_blacklist	Can use DNS names that are on the blacklist
developer	Can view additional status and diagnostic information
volume_manager	Can specify custom host paths for volumes
override_vm_priority	Can change VM scheduling priority
bypass_quotas	Bypasses all resource limits

Per-User Quotas

Quotas limit the resources a single user can consume. When a user exceeds a quota, the API returns an error and the operation is rejected.

Quotas are organized into sections in the user object (see schema above):

Section	Quota Fields	Description
vmdb	max_instances, min_memory_mb, def_memory_mb, max_memory_mb, min_vcpus, max_vcpus	Instance creation limits and memory/vCPU bounds
vmm	max_vcpus, max_memory_mb	Runtime resource limits across all running instances
net	max_service_groups, max_services, max_certificates	Networking resource limits
stor	max_volumes, min_volume_mb, max_volume_mb, max_total_volume_mb	Storage volume limits
autoscale	min_size, max_size	Autoscale group size bounds

Global Quota Overrides

Several ukpd flags set system-wide defaults and maximums that apply regardless of per-user settings:

Flag	Description
--net-user-max-service-groups	Maximum service groups per user (system limit)
--net-user-max-services	Maximum services per user (system limit)
--net-user-max-certificates	Maximum certificates per user (system limit)
--stor-vol-user-max-*	Various per-user volume limits (system limit)

The effective limit for a user is the minimum of their per-user quota and the system-wide limit.

Note: These quotas are not hard platform ceilings. Object quotas such as `max_service_groups`, `max_services`, `max_instances`, `max_certificates`, and `max_volumes` can be scaled up to very large values (hundreds of thousands per user) when needed. The practical upper bound is the performance and storage footprint of the controller's databases as object counts grow.

Reloading the User Database

The user database is loaded at controller startup from the file specified by `--users`. Changes to the file require restarting `ukp-platform.service` to take effect.

7.5 Assertions

The controller uses assertions to check internal invariants: lock state, state machine transitions, null pointers, ordering constraints, and data integrity.

Behavior	--disable-assert off (default)	--disable-assert on
	Crashes the process (SIGILL)	Logs at WARNING level and continues

Use `--disable-assert` only after being instructed to do so by Unikraft Support. Ignoring assertions can lead to data loss or corruption. Unikraft Support might tell you to set this flag if ignoring a specific assertion is harmless but allows continued operation even in the presence of a violated invariant.

7.6 Autoscaling

The autoscale engine runs inside the controller and makes scaling decisions based on proxy connection statistics and CPU metrics collected by the monitor subsystem. Autoscale policies and end-user behavior are documented in the [public documentation](#). This section covers the operator-level configuration that controls system-wide autoscale defaults and limits.

Global Autoscale Settings

Flag	Default	Description
<code>--autoscale-warmup-time-ms</code>	1000 (1s)	Default warmup time between scale-out events
<code>--autoscale-max-warmup-time-ms</code>	600000 (10 min)	Maximum warmup time users can configure
<code>--autoscale-cooldown-time-ms</code>	1000 (1s)	Default cooldown time between scale-in events
<code>--autoscale-max-cooldown-time-ms</code>	600000 (10 min)	Maximum cooldown time users can configure
<code>--autoscale-ema-alpha</code>	600	EMA smoothing factor (0–1024, fixed-point)

Autoscale Size Limits

Flag	Default	Description
--autoscale-default-min-size	0	Default minimum instances in a service group
--autoscale-default-size	1	Default target size
--autoscale-default-max-size	4	Default maximum instances
--autoscale-user-min-size	0	Minimum size users can configure
--autoscale-user-max-size	1	Maximum size users can configure

Autoscale History

The autoscale engine maintains a history of metric values for making decisions:

Flag	Default	Description
--autoscale-min-history-len	2	Minimum history entries before making decisions
--autoscale-max-history-len	8	Maximum history entries to retain

7.7 Configuration

ukpd reads configuration from two sources: 1. `/etc/ukp.conf` — Environment variables sourced by the launcher script 2. Command-line flags — Passed by the launcher script based on `/etc/ukp.conf` values

The launcher script at `/usr/lib/ukp/platform/launcher/ukpd` reads the config file, constructs the argument list, and exec's the ukpd binary.

See the Configuration Reference and Flags Reference sections below for details.

7.8 Flags Reference

This section lists all ukpd command-line flags, grouped by subsystem. Flags are passed via the launcher script based on `/etc/ukp.conf` values and the `UKPD_EXTRA_ARGS` array.

All defaults reflect the current platform version.

Root

Flag	Default	Description
-w, --workspace	/usr/local/ukp/workspace	Workspace path. Empty creates a temporary workspace.
-d, --daemonize	0	Daemonize after start
--pid-file	/usr/local/ukp/ukp.pid	Path of PID file
--delete-workspace	0	Delete workspace contents on start
--force-delete-workspace	0	Delete workspace even with running VMMs
--db-force-check	0	Force integrity check of databases on start
--db-force-update	0	Force database schema update even if in use
--db-wipe-free	0	Wipe corrupted free entries during DB checks
-c, --check-config	0	Validate configuration and exit (dry run)

Logging

Flag	Default	Description
--disable-assert	0	Disable runtime assertions
--log-format	0	Log format: 0 = file, 1 = journald
--log-path	- (stdout)	Log file path. Use - for stdout.
--log-level	1 (info)	Log level: 0 = debug, 1 = info, 2 = warning, 3 = error
--log-rotation	rotate	Log rotation mode: rotate, truncate, or append
--log-sink-config	(empty)	Path to log sink configuration JSON file

Users

Flag	Default	Description
--users	(empty)	Path to users JSON file
--no-authorization	0	Disable API authorization (development only). WARNING: Do not use in production.

Worker Pool

Flag	Default	Description
--wrk-workers	4	Number of worker threads
--wrk-interval-ms	250	Interval of periodic background jobs (ms)
--wrk-zombie-limit	10	Maximum number of zombies per object type before cleanup

API

Flag	Default	Description
--api-endpoint	127.0.0.1:45232	Frontend API endpoint
--api-new-endpoint	127.0.0.1:45235	Backend API endpoint (inter-service communication)
--api-max-ops	253	Maximum operations per API request
--api-timeout-s	10	HTTP API connection timeout (seconds)
--api-keepalive-s	60	HTTP API keep-alive timeout (seconds)
--api-max-conns	1024	Maximum API connections
--api-max-log-bytes	16383	Maximum bytes returned for log outputs
--api-root-only	0	Restrict API to root accounts only
--api-log-enable	0	Enable API request/response logging
--api-name-suffix-len	5	Random suffix length for auto-generated names

Metrics API

Flag	Default	Description
--metrics-api-endpoint	127.0.0.1:45233	Prometheus metrics API endpoint
--metrics-api-token	(empty)	Bearer token for metrics endpoint
--metrics-api-token-2	(empty)	Secondary bearer token for metrics endpoint

Networking — Core

Flag	Default	Description
--net-ip	172.16.0.0/24	Lowest IPv4 address for user networks (CIDR with subnet size)
--net-mode	isolates	Network mode: isolates or subnets

Flag	Default	Description
--net-count	0	Maximum number of hosts/isolates. 0 = network max capacity.
--net-internal-traffic	(empty)	Internal traffic allowed. Comma-separated: srvgrp, vm. Empty = no internal traffic.

Network Interfaces

Flag	Default	Description
--net-iface-preopen	0	Pre-open all TAP devices and pass FDs to VMM
--net-iface-retention-time-s	15	Seconds to retain a freed interface before re-allocation

Internal DNS (iDNS)

Flag	Default	Description
--net-idns-api-endpoint	/var/run/ukp/ukpd-idns.api	iDNS endpoint (Unix socket)
--net-idns-domain	internal.	FQDN for internal DNS names
--net-idns-ns	ns	iDNS nameserver name
--net-idns-proxy-ip	127.0.0.1	Target address for internal domains

External DNS (eDNS)

Flag	Default	Description
--net-edns-domain	unikraft.local.	FQDN for external DNS names
--net-edns-blacklist	(empty)	Path to eDNS domain blacklist file

Proxy

Flag	Default	Description
--net-proxy-api-endpoint	/var/run/ukp/ukpd-proxy.api	Proxy API endpoint (Unix socket)
--net-proxy-srv-jit-enable	0	Enable JIT (just-in-time) service creation

Services and Certificates

Flag	Default	Description
--net-user-max-service-groups	253	Maximum service groups per user
--net-user-max-services	253	Maximum services (published ports) per user
--net-user-max-certificates	8	Maximum certificates per user
--net-cert-retry-delays	1m,5m,10m,30m,1h,6h,12h,2	Certificate provisioning retry delay schedule

VM Database (VMDb)

Flag	Default	Description
--vm-max-instances	5000	System-wide maximum total instances
--vm-retention-time-s	0	Seconds to retain stopped instances before cleanup
--vm-user-max-instances	16	Maximum instances per user
--vm-user-min-memory-mb	16	Minimum guest memory (MB)
--vm-user-def-memory-mb	128	Default guest memory (MB)
--vm-user-max-memory-mb	2048	Maximum guest memory per instance (MB)
--vm-user-min-vcpus	1	Minimum vCPUs per instance
--vm-user-max-vcpus	1	Maximum vCPUs per instance

VMM (Virtual Machine Monitor)

Flag	Default	Description
--vmm-path	/usr/local/bin/firecracker	Firecracker binary path
--vmm-boost-prio	-10	Nice value for boosted VMs (e.g., during boot)
--vmm-default-prio	19	Nice value for normal VMs
--vmm-max-drain-timeout-ms	3600000 (1h)	Maximum network drain timeout (ms)
--vmm-provide-ukp-env	0	Provide UKP environment variables to kernel
--vmm-log-level	info	Log level passed to Firecracker
--vmm-console-ports	(empty)	Additional console ports: <name>:<type>[:<path>],...
--vmm-metrics-emit	0	Emit per-VM metrics to file

Flag	Default	Description
--vmm-startdata-cache-order	12 (4096 entries)	Cached startdata FDs as 2^order
--vmm-user-max-vcpus	16	Total vCPU budget per user
--vmm-user-max-memory-mb	4096	Total memory budget per user (MB)

Restart

Flag	Default	Description
--vmm-restart-delays	0,5s,10s,20s,40s,80s,160s,320s	Restart backoff delay schedule
--vmm-restart-reset-time-s	10	Seconds without crash to reset backoff
--vmm-restart-if-missing	0	On start, restart VMs whose Firecracker process is missing

Scale-to-Zero

Flag	Default	Description
--vmm-cooldown-time-ms-min	100	Minimum cooldown time (ms)
--vmm-cooldown-time-ms-def	1000 (1s)	Default cooldown time (ms)
--vmm-cooldown-time-ms-max	86400000 (1 day)	Maximum cooldown time (ms)
--vmm-notify-time-ms-min	0 (disabled)	Minimum notification time before suspend (ms)
--vmm-notify-time-ms-def	0 (disabled)	Default notification time (ms)
--vmm-notify-time-ms-max	86399999 (~1 day)	Maximum notification time (ms)
--vmm-scaletozero-rate-limit	0 (unlimited)	Maximum s2z transitions per interval
--vmm-scaletozero-rweight	1	Rate limit weight factor for restores
--vmm-scaletozero-rtime-ms	50	Assumed restore time for rate limiting (ms)
--vmm-scaletozero-rmax-delay-ms	10000 (10s)	Maximum delay from restore rate limiting (ms)

MTSS (Multi-Tier Snapshot System)

Flag	Default	Description
--vmm-mtss-enable	0,0,0	Enable MTSS tiers (comma-separated: tier0,tier1,tier2)
--vmm-mtss-soft-use-percent	50	Disk usage % threshold before MTSS starts promoting
--vmm-mtss-min-age-s	0,120	Minimum snapshot age (seconds) per tier before promotion
--vmm-mtss-interval-ms	60000 (1 min)	Interval between MTSS scans (ms)
--vmm-mtss-rate-limit	0,0 (unlimited)	Parallel MTSS transitions per tier. 0 = unlimited.
--vmm-mtss-stop-mode	normal	Stop mode for tier 3 eviction: normal, quick, or force
--vmm-mtss-detach-iface	0	Detach network interface on tier 3 eviction

Snapshots

Flag	Default	Description
--vmm-snapshot-path	snapshots	Snapshot directory (relative to workspace)
--vmm-snapshot-block-size	8192	Memory block size in snapshots (bytes)
--vmm-snapshot-prefetch-size	1048576 (1 MB)	Bytes loaded from snapshot on page faults
--vmm-initrd-map-shared	0	Share initrd between VMs (reduces memory)
--vmm-initrd-map-shared-populate	0	Share initrd with MAP_POPULATE (pre-fault pages)

Images

Flag	Default	Description
--images-path	/var/lib/ukp/images	Image storage directory
--images-import-path	(empty)	Path to JSON file for offline image import

Inline ROMs (Ephemeral Images)

Flag	Default	Description
<code>--ephemeral-images-path</code>	<code>images</code>	Directory where ephemeral images are stored (relative to workspace)
<code>--ephemeral-images-max-files</code>	<code>16</code>	Maximum number of files per inline ROM
<code>--ephemeral-images-max-size-kb</code>	<code>1024</code>	Maximum total size of an inline ROM in KB
<code>--mkfs-erofs-path</code>	<code>/usr/bin/mkfs.erofs</code>	Path to the <code>mkfs.erofs</code> binary used to package inline ROMs

Inline ROMs allow API clients to provide arbitrary file data directly in the instance creation request, without pre-packaging and pushing a ROM image to a registry. This is useful when creating instances from a template where ENV and ARGS are frozen by the template snapshot, but instance-specific data still needs to be injected.

When an inline ROM is submitted via the API, the controller:

1. Writes the provided files to a temporary directory under `<ephemeral-images-path>/<UUID>/`
2. Calls `mkfs.erofs` to package the files into an EROFS image at `<UUID>.erofs`
3. Attaches the resulting image as a ROM device to the instance
4. Deletes the EROFS file when the last reference to it is closed (reference-counted cleanup)

The `--ephemeral-images-max-files` and `--ephemeral-images-max-size-kb` flags limit what API clients can submit per inline ROM. The `mkfs.erofs` binary must be installed on the host; the `erofs-utils` package provides it.

Limits

Limit	Default	Description
ROMs per instance	<code>8</code>	Maximum number of ROM entries (inline or image-based) per instance. Can be increased if needed.
Files per inline ROM	<code>16</code>	Configured via <code>--ephemeral-images-max-files</code> . Prevents abuse from large numbers of trivially small files.
Total size per inline ROM	<code>1 MB</code>	Configured via <code>--ephemeral-images-max-size-kb</code> . Applies across all files in a single ROM, not per file.
Max request body	<code>8 MB</code>	Proxy configuration (<code>client_max_body_size</code>). Since inline ROM data is base64-encoded in the JSON request body, this is the practical upper bound on total inline ROM data per API request.

Storage

Flag	Default	Description
--stor-vol-path	volumes	Volume storage directory (relative to workspace)
--stor-vol-tool	/usr/local/bin/ukp-vol	Path to ukp-vol tool
--stor-fs-config-path	/usr/lib/ukp/platform/config	Filesystem configuration templates directory
--stor-vol-def-fs	ext4	Default volume filesystem: ext4 or virtiofs
--stor-vol-disable-discard	0	Disable TRIM/discard for block devices
--stor-max-async-ops	16	Maximum parallel filesystem operations
--stor-squota-uid	0	UID for static quota enforcement
--stor-squota-gid	0	GID for static quota enforcement
--stor-dquota-gid	1000000	GID for dynamic quota enforcement
--stor-max-volumes	5000	System-wide maximum number of volumes
--stor-max-total-volume-mb	1048576 (1 TB)	System-wide total volume storage budget (MB)
--stor-user-max-volumes	253	Maximum volumes per user
--stor-user-min-volume-mb	8	Minimum volume size (MB)
--stor-user-max-volume-mb	1024	Maximum volume size (MB)
--stor-user-max-total-volume-mb	1024	Maximum total volume size per user (MB)

Autoscale

Flag	Default	Description
--autoscale-warmup-time-ms	1000 (1s)	Default warmup time between scale-out events (ms)
--autoscale-max-warmup-time-ms	600000 (10 min)	Maximum configurable warmup time (ms)
--autoscale-cooldown-time-ms	1000 (1s)	Default cooldown time between scale-in events (ms)
--autoscale-max-cooldown-time-ms	600000 (10 min)	Maximum configurable cooldown time (ms)
--autoscale-ema-alpha	600	EMA smoothing factor (0-1024, fixed-point)
--autoscale-min-history-len	2	Minimum metric history entries before making decisions
--autoscale-max-history-len	8	Maximum metric history entries to retain

Flag	Default	Description
--autoscale-default-min-size	0	Default minimum autoscale group size
--autoscale-default-size	1	Default autoscale group size
--autoscale-default-max-size	4	Default maximum autoscale group size
--autoscale-user-min-size	0	Minimum group size users can configure
--autoscale-user-max-size	1	Maximum group size users can configure

HAC (Host Agent Client)

Flag	Default	Description
--hac-enable	0	Enable HAC connection to the agent
--hac-api-endpoint	/var/run/ukp/agent.api	HAC Unix socket path

Cluster

Node

Flag	Default	Description
--cl-node-name	(empty = hostname)	Node name. Empty uses the system hostname.
--cl-node-roles	(empty = standalone)	Node roles: vmhost, controller, leader. Empty = standalone mode.

Cluster API

Flag	Default	Description
--cl-api-endpoint	127.0.0.1:45234	Cluster API endpoint (inter-node communication)
--cl-controller	(empty)	Address of cluster controller node
--cl-api-timeout-s	10	Backend API connection timeout (seconds)
--cl-api-keepalive-s	300 (5 min)	Backend API keep-alive timeout (seconds)
--cl-api-max-conns	1024	Maximum backend connections per peer
--cl-api-cache-conns	5	Maximum cached client connections

8 Agent

The agent is a companion process to the platform controller that handles tasks outside the domain of launching VMs, such as managing images, provisioning TLS certificates through the ACME protocol, exporting metrics and logs. It also serves the system health check.

Packaged as `ukp-agent`, it starts after `ukp-platform`. The two components communicate over a Unix domain socket using the HAC (Host-Agent-Communication) protocol.

The agent consists of distinct subsystems for handling those auxiliary tasks:

8.1 Image Management

The agent manages the images used to spawn VMs under `/var/lib/ukp/images`. For more information about the images themselves, see [Images](#).

The agent is configured with a *canonical registry* (via `AGENT_CANONICAL_REGISTRY_HOST` and `AGENT_CANONICAL_REGISTRY_TOKEN`) as the source of images.

Multiple distribution modes are supported:

Dynamic Image Pull

When a user creates an instance referencing an image not yet available on the host, `ukpd` sends a pull request to the agent via HAC.

This behavior is **automatic** when the agent is running — no additional flags are required.

The `AGENT_PULL_INCLUDE` and `AGENT_PULL_EXCLUDE` environment variables on the agent control which image name patterns are eligible for dynamic pulling.

The agent fetches the image from the *canonical registry* and `ukpd` resumes instance creation.

User experience: If the requested image is not cached locally, the user experiences a delay on the first instance creation while the image is pulled. Subsequent creations of the same image use the local cache.

Direct push

Enabled with `AGENT_REGISTRY_ENABLE`, the agent runs a OCI registry, allowing users to push images to `index.<metro>` directly directly.

Sticky images

Sticky images are images that must always be present on the node. They are configured via `AGENT_CANONICAL_REGISTRY_DEFAULT_IMAGES`.

When referencing an image by tag here, it is resolved and the latest reference available in the registry is pulled.

The agent's health checker monitors sticky images and reports **degraded** status if any are missing from local storage. This ensures that critical base images are always available for instance creation without requiring an on-demand pull.

Syncing from canonical registry (Harbor)

The agent will subscribe to NATS events when new images are pushed and pull them, making them available to ukpd.

This functionality is enabled with `AGENT_HARBOR_ENABLE` and requires the following configuration:

Environment	Purpose
<code>AGENT_HARBOR_NATS_URL</code>	URL of the NATS server delivering <code>harbor.artifact.pushed</code> events
<code>AGENT_HARBOR_NATS_USER</code>	Credential to use when authenticating with the NATS server
<code>AGENT_HARBOR_NATS_PASSWORD</code>	-

When pulling from the registry the *canonical registry* configuration is used to authenticate pulls.

HTTP Image Transport

While the default transport for distributing images follows the OCI distribution-spec, we also support plain HTTP endpoints when fetching images.

The image is referenced as `http+oci://example.com/<user>/<image>/<tag-or-digest>`.

The agent always resolves tags to digests when fetching. When requesting an image tag like `http+oci://example.com/<user>/<image>/latest` the server must respond with a **relative** path to a digest under the same namespace: `http+oci://example.com/<user>/<image>/@<digest>`.

The response should be identified as `application/vnd.oci.layout.v1+tar` and must be a TAR archive containing an [OCI image layout](#).

8.2 TLS Certificate Provisioning (ACME)

The agent implements an ACME v2 client for automated TLS certificate provisioning from Let's Encrypt (or other ACME-compatible CAs).

The subsystem is enabled by default and can be controlled via `AGENT_ENABLE_CERT_SERVICE`.

When ukpd needs a certificate for a service domain:

1. ukpd sends a certificate request via HAC
2. The agent creates/reuses an ACME account
3. The agent completes the ACME HTTP-01 challenge (webroot-based, default `/var/www/html`)
4. The agent receives the certificate and sends it back to ukpd
5. ukpd stores the certificate in CERTDB

Challenge types

Only **HTTP-01** (webroot) is supported. DNS-01 is not implemented in the agent code-base. The agent places challenge tokens under the configured webroot directory (default `/var/www/html/.well-known/acme-challenge/`).

Custom CA support

The agent can target any ACMEv2-compatible CA (not just Let's Encrypt):

Environment	Purpose
AGENT_ACME_DIR_URL	ACME directory URL. Point to Step-CA, Pebble, or any ACMEv2 CA for on-prem deployments.
AGENT_ACME_INSECURE_	Skip TLS verification when talking to the ACME CA. Required when the CA uses self-signed certificates.
AGENT_ACME_WEBROOT	Override the default webroot directory for HTTP-01 challenge tokens.

Renewal

- Certificates are checked for renewal every **7 days**.
- The agent notifies the platform **14 days** before expiration.
- There is no built-in retry on failure — a failed renewal is re-attempted on the next check period (7 days later).

8.3 OpenTelemetry Metrics Export

The agent embeds Prometheus `node_exporter` that collects host-level system metrics. The exporter exposes metrics on an authenticated endpoint (bearer token via `AGENT_PROMETHEUS_BEARER_TOKEN`).

This complements the per-VM metrics emitted by ukpd's VMM subsystem.

8.4 API Log Proxying

The agent can forward API request/response logs from ukpd to an OpenTelemetry collector. ukpd sends log entries via HAC, the agent formats and ships them to the configured endpoint.

The endpoint and authentication can be configured using `AGENT_OTEL_COLLECTOR_TOKEN` and `AGENT_OTEL_COLLECTOR_TOKEN`.

8.5 Health checking

When not disabled (via `AGENT_HEALTH_SERVICE_ENABLE`) the agent provides the `/v1/healthz` end-point reporting on the overall system health. It currently runs internal health checks for platform services and Sticky Images.

9 Virtual Machine Monitor (Firecracker)

The Virtual Machine Monitor (VMM) subsystem is responsible for creating, running, and managing the MicroVMs that host application instances on the platform. Each application runs inside its own Firecracker process, providing hardware-level isolation between tenants while maintaining the fast startup times that make scale-to-zero practical. The VMM subsystem of ukpd orchestrates the lifecycle of these MicroVMs, handling operations like booting, suspending, resuming, and shutting down instances based on platform events and traffic patterns.

9.1 Firecracker Integration

Firecracker is packaged as `ukp-firecracker`. The binary path is configurable via `--vmm-path` (default: `/usr/bin/firecracker`).

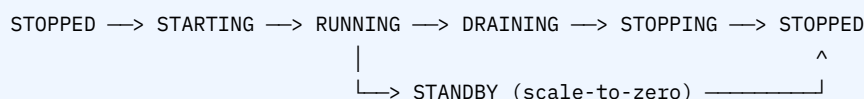
For each VM, ukpd:

1. Writes a new configuration file if necessary
2. Generates a new startdata blob if necessary
3. Forks a child process
4. Exec's Firecracker, keeping a pidfd to the Firecracker process

ukpd monitors the health of each Firecracker process and updates the state if there is any change. If a Firecracker process crashes, ukpd detects it and applies the configured restart policy.

9.2 VM Lifecycle

A VM transitions through these states:



State	Description
STOPPED	Not running. No resource consumption.
STARTING	Booting (typically milliseconds).
RUNNING	Application is executing.
DRAINING	Existing connections are draining.
STOPPING	Shutting down.
STANDBY	Scaled to zero. ukpd will wake VM for incoming traffic

A VM can also be a **template**, which is a snapshot used to clone new instances.

Stop reasons, drain behavior, and restart policies are configured via runtime flags.

9.3 Priority Boost

The priority boost feature allows fresh Firecracker processes to have a higher OS scheduling priority over the existing Firecracker processes during startup. This can help reduce cold start latency under high load. This feature can be configured via the `ukpd --vmm-boost-prio` and `--vmm-default-prio` flags.

9.4 Startdata

When booting a VM, `ukpd` serializes a per-VM configuration blob called **startdata** and passes it to Firecracker as a memfd. Firecracker exposes this data via a virtual memory-mapped I/O device to the guest, which in turn exposes it as a JSON blob to the user-space. The guest init system (tinyx) reads this data to configure the VM at boot and reapplies the configuration on resume from standby.

Startdata blobs are cached to avoid recomputation on repeated starts. The cache uses sharding with random eviction and defaults to 4096 entries. `ukpd` automatically invalidates a cache entry when a VM's configuration changes. Cache size is controlled by `--vmm-startdata-cache-order`.

Format

The startdata in the guest is exposed as JSON with the following format:

Key	Contents
mac	MAC address of the guest
ip	IP address of the guest in CIDR notation
gw	Gateway IP address for the guest
hostname	Host name for the guest

This list should be considered non-exhaustive as additional fields may be added in the future to support new features.

9.5 Snapshots

Firecracker supports snapshotting VM memory and device state to disk. `ukpd` uses this for: - **Stateful scale-to-zero** — Save VM state before suspending; restore on wake-up - **Templates** — Create a snapshot that serves as a base image for cloning new instances

Snapshot files are stored in the `ukpd` workspace under the `snapshots` directory. The location is configurable by the `--vmm-snapshot-path` `ukpd` flag. The snapshot format uses LZ4 compression and only stores non-zero pages to minimize disk usage. When resuming and then suspending a VM, the existing snapshot will be updated with only the modified memory pages.

See [Snapshot Management](#) for the Multi-Tier Snapshot System that manages snapshot life cycle.

9.6 Suspend and Resume

When the platform decides to suspend a VM, it sends a suspend signal to the VMM, which triggers the creation of a snapshot and the exit of the Firecracker process. The result is a **snapshot**, the on-disk artifact consisting of two files:

File	Contents
<UUID>.state	Serialized device state (vCPU registers, virtio queues, etc.)
<UUID>.mem	Guest memory contents

Suspend Abort

If new traffic arrives while suspension is in progress, the platform can send an abort signal to cancel the snapshot and keep the VM running, avoiding unnecessary cold-boot latency for the incoming request. This signal is a best-effort attempt and can be ignored by the VMM if an abort is not possible anymore.

9.7 Console

ukpd can expose virtio-console ports from the VM to the host. This is configured via `--vmm-console-ports` in ukpd.

Configuration format

```
<name>:<type>[/<format>][:<path>],...
```

Component	Values	Description
name	any string	Logical name for the port. A port named <code>default</code> is auto created as <code>stdout</code> if no ports are configured.
type	<code>stdout</code> , <code>file</code> , <code>socket</code>	Output destination. <code>stdout</code> writes to the VM's standard output (no path needed). <code>file</code> writes to a file on the host (path required). <code>socket</code> connects to a UNIX socket (path required).
format	<code>raw</code> (default), <code>json</code>	Output format. <code>json</code> is only supported with <code>socket</code> type.
path	filesystem path	Required for <code>file</code> and <code>socket</code> types.

JSON format

The JSON format uses line-buffering to emit a JSON record for every line written in the guest. The record format uses the following structure:

Key	Description
name	Name of the console port
uuid	UUID of the VM emitting the message
message	The line emitted by the VM

The message has a maximum line length of 16 KiB. If a larger line is written to a JSON formatted console, an error structure will be emitted.

Examples

Configuration	Behavior
default:stdout	VM serial output goes to standard output (written to <code>vm.log</code>).
app:file:/var/log/ukp/app.log	Dedicated logfile for application output.
monitor:socket/json:/var/run/ukp/vm-monitor.sock	JSON-formatted output to a UNIX socket for external monitoring tools.

Multiple ports are comma-separated:

```
default:stdout,debug:file:/var/log/ukp/debug.log,events:socket/json:/var/run/ukp/events.sock
```

9.8 ukpd Communication

Firecracker notifies ukpd of VM events via a POSIX message queue (`ukp-vmm-notify`). Events that are sent over this queue:

- Boot completion
- Port announcements
- Volume usage
- Template signaling

There is also a UNIX domain socket for bidirectional communication between ukpd and Firecracker. For communication, destined to the VM, messages are forwarded over a vsock channel to the guest. This channel delivers:

- Scale-to-zero notifications to the guest

9.9 Metrics

The VMM subsystem can emit per-VM metrics via `--vmm-metrics-emit`. When enabled, Firecracker writes prometheus-style metrics into the specified file.

9.10 Logging

ukpd manages the logfiles and creates them in the corresponding instance directory in the ukpd workspace. The VMM writes VM logs from the serial console and default [Console](#) port to `vm.log`, and VMM logs to `vmm.log`. The log level of the VMM itself is configurable via `--vmm-log-level`.

9.11 Key Flags In ukpd

Flag	Default	Description
<code>--vmm-path</code>	<code>/usr/bin/firecracker</code>	Path to Firecracker binary
<code>--vmm-boost-prio</code>	<code>-10</code>	Nice value for boosted VMs
<code>--vmm-default-prio</code>	<code>19</code>	Nice value for normal VMs
<code>--vmm-max-drain-timeout-ms</code>	<code>3600000 (1h)</code>	Maximum connection drain time
<code>--vmm-restart-delays</code>	<code>0,5s,10s,20s,40s,80s,160s,864*5m</code>	Restart backoff schedule
<code>--vmm-restart-reset-time-s</code>	<code>10</code>	Time without crash to reset backoff
<code>--vmm-cooldown-time-ms-def</code>	<code>1000 (1s)</code>	Default scale-to-zero cooldown
<code>--vmm-mtss-enable</code>	<code>0,0,0</code>	Enable MTSS tiers (3 values)
<code>--vmm-console-ports</code>	<code>(empty string)</code>	Console port configuration
<code>--vmm-metrics-emit</code>	<code>0</code>	Enable per-VM metric emission
<code>--vmm-snapshot-path</code>	<code>snapshots</code>	Snapshot storage path (relative to workspace)
<code>--vmm-log-level</code>	<code>info</code>	VM log verbosity

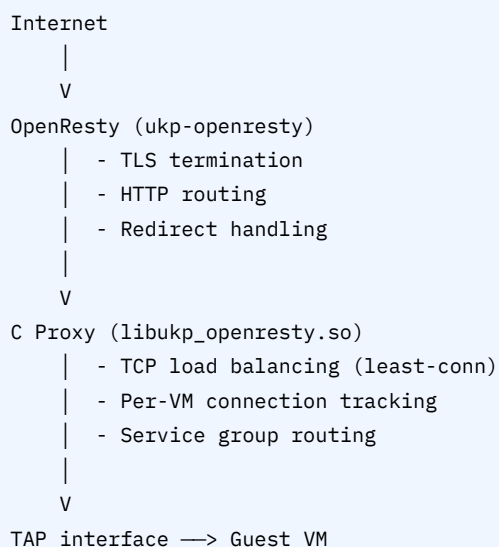
See the [Flags Reference](#) for the complete list.

10 Proxy

The proxy is the entry point for all inbound traffic to application instances running on the platform. Every HTTP request, WebSocket connection, and TCP stream from the outside world passes through the proxy before reaching a MicroVM. The proxy handles TLS termination, routes requests to the correct service group, balances load across instances, and triggers scale-to-zero wake-ups when traffic arrives for a suspended instance. It is also the source of the connection statistics that drive autoscaling decisions.

The platform uses a two-layer architecture: OpenResty for HTTP/TLS termination and the C proxy for TCP load balancing.

10.1 Architecture



10.2 OpenResty Layer

OpenResty (NGINX + LuaJIT) handles:

- **TLS termination** using metro wildcard-certificate or custom service group certificates
- **HTTP mode** load balancing at the request level
- **Redirect handling** (e.g., HTTP→HTTPS redirects on port 80)

The launcher script generates the OpenResty configuration from a template (`nginx.conf.template`) and uses Lua scripts for runtime routing decisions:

Lua Script	Purpose
<code>init.lua</code>	Per NGINX-worker initialization
<code>access.lua</code>	HTTP request routing

Lua Script	Purpose
<code>preread.lua</code>	TLS SNI (Server Name Indication) routing
<code>balancer.lua</code>	Upstream selection based on routing selection
<code>preread_internal.lua</code>	Routing for internal service groups
<code>cert.lua</code>	Dynamic TLS certificate selection
<code>log.lua</code>	Per-request or connection cleanup
<code>upstream_notify.lua</code>	Hooks for statistics

10.3 C Proxy Layer

The C proxy is built as a shared library (`libukp_openresty.so`) loaded by OpenResty. It handles the core routing functionality and communication with `ukpd`.

Load Balancing

The C proxy implements a **least-connections** algorithm. For each incoming connection/request, it:

1. Iterates over all instances in the service group
2. Finds the set of running instances
3. Selects from that set the instance with the fewest in-flight connections (TCP mode) or requests (HTTP mode)

If there is no suitable VM, it can request waking up an existing VM. Or it can request a new VM for service groups with an on-demand autoscale policy.

Connection Tracking

The proxy tracks per-VM connection counts and reports statistics to `ukpd` via POSIX message queues. These stats feed into:

- The autoscale engine (for scale-out/scale-in decisions)
- The scale-to-zero system (for idle detection)

Communication with `ukpd`

The C proxy communicates with `ukpd` through a combination of UNIX domain sockets and shared memory. For asynchronous notifications, it uses POSIX message queues.

10.4 Server-Timing Header

The proxy can include a `Server-Timing` header in HTTP responses, providing timing breakdowns visible in browser developer tools.

Enabled via the `UKP_PROXY_SERVER_TIMING_HEADER` environment variable (any non-empty value enables it). The header name is configurable via `UKP_PROXY_SERVER_TIMING_HEADER_NAME` (default: `Server-Timing`).

The header provides a timing breakdown per the [Server-Timing specification](#):

Metric	Description
<code>proxy_srvgrp_fetch</code>	Service group lookup time if not cached
<code>ukpd_ipc_recv</code>	IPC receive time in ukpd
<code>ukpd_vm_create</code>	VM creation time
<code>ukpd_vm_start_prereq_time</code>	Time until all pre-requisites are met
<code>ukpd_vmm_prestart</code>	Time until VMM is about to be execed
<code>ukpd_vmm_start</code>	VMM main reached
<code>ukpd_vmm_load</code>	VMM image/initrd load time
<code>ukpd_vmm_ready</code>	VMM ready signal time
<code>ukpd_vmm_boot</code>	VMM guest boot time
<code>ukpd_vmm_net</code>	VMM first port open time
<code>ukpd_ipc_send</code>	IPC response send time from ukpd
<code>proxy_vm_wait</code>	Proxy waiting for VM readiness
<code>proxy_forward</code>	Proxy start forwarding time
<code>nginx_upstream_header</code>	NGINX first upstream response header time

10.5 Instance UUID and Node ID Header

The proxy can include a custom HTTP response header identifying which metro node served the request. This is useful for monitoring in multi-node deployments.

Enabled via the `UKP_PROXY_NODE_ID_HEADER` environment variable. The header name is configurable via `UKP_PROXY_NODE_ID_HEADER_NAME` (default: `Node-Id`). The value is the cluster node UUID from the HAC CONNECT handshake.

Additionally, an `Instance-Id` header is available via the `UKP_PROXY_INSTANCE_ID_HEADER` environment variable and the header name is configurable via `UKP_PROXY_INSTANCE_ID_HEADER_NAME` (default header name: `Instance-Id`).

10.6 Service Groups

Service group configuration (domains, ports, certificates, autokill) is documented in the [public documentation](#).

10.7 Certificate Management

Certificate management is documented in the [public documentation](#).

10.8 Configuration

Key proxy-related flags:

Flag	Description
<code>--net-proxy-api-endpoint</code>	Proxy API endpoint address
<code>--net-proxy-srv-jit-enable</code>	Enable JIT service creation
<code>--net-user-max-service-groups</code>	Maximum service groups per user
<code>--net-user-max-services</code>	Maximum services (published ports) per user
<code>--net-user-max-certificates</code>	Maximum certificates per user
<code>--net-cert-retry-delays</code>	Retry delays for certificate provisioning

11 Snapshot Management

Snapshots are the mechanism that enables stateful scale-to-zero on the platform. When an instance scales to zero, its full memory and device state are written to disk as a snapshot. When traffic arrives again, the platform restores the snapshot and resumes the instance exactly where it left off — without a cold boot. Managing these snapshots efficiently is critical for deployments with large numbers of instances, because uncompressed snapshots consume significant disk space. The Multi-Tier Snapshot System (MTSS) solves this by automatically promoting snapshots through compression and deletion tiers based on age and disk pressure, allowing the platform to scale to hundreds of thousands of instances per host without running out of storage.

11.1 MTSS (Multi-Tier Snapshot System)

MTSS manages snapshot lifecycle to balance performance and disk usage. It operates on three tiers:

Tier	Storage	Description
Tier 0	Uncompressed	Fastest restore. Snapshots are kept as-is on disk.
Tier 1	Compressed	Snapshots are LZ4-compressed to reduce disk usage. Slightly slower restore due to decompression.
Tier 2	Deleted	Snapshots are removed entirely. The instance will cold-boot on next wake-up.

MTSS periodically evaluates snapshots and promotes them through tiers based on age and disk pressure. Snapshots can start in Tier 1 (uncompressed) or Tier 2 (compressed).

How it works

1. When a VM is suspended, a Tier 0 (uncompressed) or a Tier 1 (compressed) snapshot is created (depending on the configuration)
2. MTSS promotion runs at a configurable interval (`--vmm-mtss-interval-ms`)
3. If disk usage exceeds the soft threshold (`--vmm-mtss-soft-use-percent`), MTSS begins promoting snapshots
4. Older snapshots (exceeding `--vmm-mtss-min-age-s`) are moved from Tier 0 to Tier 1 (compressed)
5. If disk pressure continues, Tier 1 snapshots are moved to Tier 2 (deleted)
6. When a snapshot is deleted, the `--vmm-mtss-stop-mode` controls how the associated VM is stopped (normal, quick, or force)

Note: With the `ukpd-managed snapshots` feature (staging), snapshots will by default directly start in Tier 1 (compressed); MTSS configuration: `0,1,0`

Configuration

MTSS supports fine-grained configuration.

Flag	Default	Description
<code>--vmm-mtss-enable</code>	<code>0,0,0</code>	Enable MTSS tiers. Three comma-separated values for Tier 0, 1, 2.
<code>--vmm-mtss-soft-use-percent</code>	50	Disk usage percentage that triggers snapshot promotion
<code>--vmm-mtss-min-age-s</code>	<code>0,120</code>	Minimum age (seconds) before a snapshot can be promoted. Two comma-separated values: tier 0→1 (0s) and tier 1→2 (120s).
<code>--vmm-mtss-interval-ms</code>	60000 (1 min)	How often MTSS evaluates snapshots
<code>--vmm-mtss-rate-limit</code>	<code>0,0</code> (unlimited)	Maximum number of promotions per interval
<code>--vmm-mtss-stop-mode</code>	normal	Stop mode when deleting a snapshot's VM (normal/quick/force)
<code>--vmm-mtss-detach-iface</code>	0	Whether to detach network interfaces when stopping a VM during MTSS eviction

Set `--vmm-mtss-enable` to `0,0,0` to only use uncompressed snapshots, `0,1,0` to directly use compressed snapshots only, `1,1,0` to start with uncompressed snapshots and then promote them to compressed snapshots (without the final deletion step), or `1,1,1` for the full cycle (uncompressed, compressed, delete).

Tuning Guidance

For latency-sensitive workloads: Enable Tier 0 and Tier 1 but disable Tier 2 (`--vmm-mtss-enable 1,1,0`). This ensures snapshots are always available for fast restore, trading disk space for consistent wake-up latency.

For cost-optimized deployments: Enable all three tiers or Tier 1 and Tier 2 only (`--vmm-mtss-enable 1,1,1` or `0,1,1`). Infrequently accessed instances will have their snapshots deleted, freeing disk space at the cost of cold-boot latency on wake-up.

Balanced setup: Enable only Tier 1 (`--vmm-mtss-enable 0,1,0`). The new default (staging). This will directly generate compressed snapshots but never deletes them. Generating a compressed snapshot causes slightly more CPU load but saves disk space.

Disk usage threshold: The `--vmm-mtss-soft-use-percent` flag controls when MTSS begins reclaiming space. A lower value is more aggressive (starts reclaiming earlier); a higher value allows more snapshots to persist.

Generally, snapshotting uses a minimal delta strategy. The VMM uses deferred block-wise loading for snapshot memory, with configurable prefetching (`--vmm-snapshot-prefetch-size`).

The snapshotting process is incremental, meaning the platform updates only the blocks in the snapshot file that have changed since the last snapshot (i.e., during the VM's current run cycle). Additionally, compressed blocks untouched by the VM in the current run cycle remain compressed in the snapshot file, even during uncompressed snapshot runs (Tier 0). Thus, transitioning from Tier 0 to Tier 1 requires compressing only the touched (uncompressed) blocks.

Related flags:

Flag	Default	Description
<code>--vmm-snapshot-block-size</code>	8192 bytes	Block size for snapshot I/O
<code>--vmm-snapshot-prefetch-size</code>	1048576 bytes (1 MiB)	Prefetch size for snapshot restore (affects wake-up performance)

11.2 Snapshot Storage

Snapshots are stored in the workspace directory, typically under a per-VM subdirectory. The path is configurable via `--vmm-snapshot-path`.

11.3 Snapshot orchestration

With ukpd-managed snapshots (staging), the platform controller directly orchestrates snapshotting. This enables improved throughput control and fast snapshot cancelation in case new traffic for the VM arrives during the snapshotting process. Previously, snapshots were made independently by the individual VMM processes (legacy engine).

To disable ukpd-managed snapshots and revert to the legacy snapshot engine, use the `--vmm-disable-ukpd-managed-snapshots` option (recommended only in exceptional cases). This mode is incompatible with the direct creation of compressed snapshots (`--vmm-mtss-enable 0,1,0` and `0,1,1`).

The platform controller dispatches snapshot creation and delayed MTSS compression workloads to a distinct worker pool. The pool will dynamically scale up and down the number of workers depending on the current load. With `--vmm-snapshot-max-workers` you can fine-tune the maximum threads in the pool. Note that higher values do not necessarily speed up snapshot creation and may lead to memory contention.

By default, the controller will try to use transparent hugepages for mapped snapshot files during snapshot operations. You can disable this with `--vmm-vmm-snapshot-disable-hugepages`.

11.4 Scale-to-Zero Wake-Up Flow

When a request arrives for a scaled-to-zero instance, the platform executes a wake-up chain that completes within a single round-trip time. The proxy buffers the incoming request, the controller resumes the instance via the VMM, and once the instance signals ready, the buffered request is

forwarded. Understanding this flow is important for diagnosing wakeup latency and tuning rate limiting.

11.5 Scale-to-Zero Rate Limiting

Rate limiting prevents thundering herd problems during mass scale-to-zero.

Formula: $n = (n_{\text{restores}} \times r_{\text{weight}}) + n_{\text{suspends}}$ - If $n \geq \text{rate_limit}$, the suspend is postponed - After $r_{\text{max_delay_ms}}$ past the original deadline, restores are excluded ($n = n_{\text{suspends}}$ only), guaranteeing eventual suspend - Each restore creates a slot lasting $r_{\text{time_ms}}$, then n_{restores} decrements - Only applies to stateful (suspend) operations, not stateless stop - If $r_{\text{weight}}=0$, restores never count

Configuration

Flag	Default	Description
<code>--vmm-scaletozero-rate-limit</code>	0 (unlimited)	Max s2z transitions per interval
<code>--vmm-scaletozero-rweight</code>	1	Weight factor for restores
<code>--vmm-scaletozero-rtime-ms</code>	50	Assumed restore time (ms)
<code>--vmm-scaletozero-rmax-delay-ms</code>	10000 (10s)	Max delay from rate limiting

12 Networking

Networking is a foundational layer of the platform that provides each MicroVM with its own isolated network connection to the outside world. Every instance gets a dedicated TAP network interface with its own IP address, and the platform manages the full lifecycle of these interfaces — from allocation and IP assignment through to firewall rule enforcement and DNS resolution. The networking subsystem ensures that tenants are isolated from each other at the network level, that VMs can resolve both internal and external DNS names, and that the firewall enforces traffic policy without requiring manual operator intervention.

12.1 Firewall

The platform manages firewall rules on each host to enforce network isolation and security. Two components are involved:

- **ukp-firewall.service** — A systemd service that initializes iptables rules, ipsets, and NAT configuration at boot. Managed by the `ukp-networking` package.
- **ukpd** — The platform controller dynamically manages nftables set elements to enforce VM-to-VM isolation based on user ownership.

ukp-firewall.service

The `ukp-firewall.service` is a oneshot systemd service that configures the host firewall. It is not invoked directly — operators manage it through systemd:

Command	Behavior
<code>systemctl start ukp-firewall</code>	Creates ipsets (if they don't exist), flushes and rebuilds all iptables rules, enables IP forwarding and NAT.
<code>systemctl stop ukp-firewall</code>	Disables IP forwarding, flushes all iptables rules, destroys all ipsets.
<code>systemctl restart ukp-firewall</code>	Calls stop then start. Ipsets are destroyed and recreated.
<code>systemctl reload ukp-firewall</code>	Flushes and rebuilds iptables rules but preserves ipsets . Use this to apply firewall customizations without losing ipset entries populated by <code>ukp-vnet.service</code> .

Note: The `reload` action is mostly idempotent, but ipsets are not flushed. This design allows `ukp-firewall.service` to be reloaded independently of `ukp-vnet.service`, which populates the ipsets with customer network information.

Startup Sequence

When the firewall service starts, it performs the following steps:

1. **Load configuration** — Reads `/etc/ukp.conf` to obtain `NET_IFACE`, `NET_SEGMENT`, and other network settings.
2. **Detect network interface** — If `NET_IFACE` is empty, the interface associated with the default route is detected automatically. The service retries up to 5 times (with 5-second delays) if the default gateway is not yet available.
3. **Determine IP addresses** — Retrieves the IPv4 address of the public-facing interface. For IPv6, the first non-link-local address is used. The service retries if the interface is not yet configured.
4. **Create ipsets** — Creates the three ipsets (`ukp.net4`, `ukp.gw4`, `ukp.block4`) if they do not exist, and initializes `ukp.block4` with reserved address ranges.
5. **Flush iptables** — Clears all existing rules from `filter`, `nat`, `mangle`, and `raw` tables (both IPv4 and IPv6).
6. **Set default policies** — Sets `INPUT`, `OUTPUT`, and `FORWARD` chains to `DROP` by default.
7. **Execute early hooks** — Runs scripts in `/etc/ukp.d/firewall/early.d/`.
8. **Install main rules** — Configures NAT/masquerading for guest traffic, TPROXY rules for internal service group traffic, DNS access rules, and forwarding allowances.
9. **Execute late hooks** — Runs scripts in `/etc/ukp.d/firewall/late.d/`.
10. **Enable IP forwarding** — Sets `net.ipv4.ip_forward=1`.

Ipsets

The firewall creates and manages three ipsets that are referenced by iptables rules:

Ipset	Type	Purpose
<code>ukp.net4</code>	<code>hash:net,iface</code>	Customer networks paired with their TAP interfaces. Populated by <code>ukp-vnet.service</code> .
<code>ukp.gw4</code>	<code>hash:net,iface</code>	Gateway addresses (host side) paired with their interfaces. Populated by <code>ukp-vnet.service</code> .
<code>ukp.block4</code>	<code>hash:net</code>	Blocked destination networks. Pre-populated with reserved IP ranges to prevent VMs from reaching private networks. This set is intended to be customized by operators.

The `ukp.block4` set is initialized with addresses from [IANA reserved ranges](#) including loopback (`127.0.0.0/8`) and local networks (`0.0.0.0/8`). Additional private ranges (RFC 1918, carrier-grade NAT, link-local, test networks) are added by the `10-block-private4` hook but can be customized by operators.

Firewall Customization

The firewall supports customization through hook scripts executed at specific points during startup. Two hook directories are available:

Directory	Timing
/etc/ukp.d/firewall/early.d/	Executed before the main firewall rules are installed. Rules added here take precedence.
/etc/ukp.d/firewall/late.d/	Executed after the main rules are applied. Use for appending additional logic.

Only executable files in these directories are invoked. Each hook receives the following arguments:

```
./hook $ACTION $NET_IFACE_OUTBOUND $ADDR4_IFACE $ADDR6_IFACE
```

Additionally, hooks have access to environment variables including \$HAVE_IP6, \$NET_IFACE, \$NET_SEGMENT, and the ipset names (\$NET_NET4_SET, \$NET_GW4_SET, \$NET_BLOCK4_SET).

The ukp-networking package installs default hooks:

Hook	Directory	Purpose
10-block-private4	early.d/	Adds RFC 1918 private networks and other reserved ranges to the ukp.block4 ipset.
20-allow-ssh	early.d/	Allows SSH access to the host. Configurable via NET_IFACE_SSHD_LISTEN in /etc/ukp.conf.
20-allow-icmp-reply	early.d/	Allows ICMP echo requests (ping) to the public IP address. Not enabled by default.

Note: Hook scripts must check the \$ACTION argument for forward compatibility. Currently only start is used.

ukpd Nftables Integration

In addition to the iptables rules managed by ukp-firewall.service, ukpd dynamically manages nftables set elements to enforce VM-to-VM isolation based on user ownership.

Architecture

- ukpd uses nftables via the **libmnl/libnftnl** C API
- Family: ip (IPv4 only, NFPROTO_IPV4)
- Table: filter (pre-provisioned on the host by ukp-firewall.service)
- ukpd does **not** create tables, chains, or rules — it only manages **set elements**

Managed Sets

ukpd manages two nftables sets that map VM IP addresses to user identifiers:

Set	Purpose
ukp.ip2uid	Maps IPv4 addresses to user IDs
ukp.uid2ip	Maps user ID + IPv4 tuples for reverse lookup

The external nftables rules (not managed by ukpd) use these sets to: - **ALLOW** VM-to-VM traffic between VMs owned by the same user - **DENY** VM-to-VM traffic between different users

Lifecycle

- **VM start:** ukpd adds the IP-to-user mapping to both sets atomically
- **VM stop:** ukpd removes the IP-to-user mapping from both sets
- **ukpd start:** flushes and repopulates both sets from VMDB
- **Enable condition:** `--net-internal-traffic` must include `vm`

Operator Custom Rules

Operators can freely add custom rules to the `filter` table or any other table without conflicting with ukpd, since ukpd never touches rules or chains — only set elements.

Error Handling

If any nftables operation fails, ukpd enters a restricted secure mode to avoid breaking user isolation. **No new mappings can be added** until ukpd is restarted. This effectively disables VM creation. Deletions are always attempted regardless of the error state.

12.2 Dependencies

The firewall subsystem requires: - `libmnl0` — Minimalistic Netlink library - `libnftnl11` — Netfilter nftables userspace library

These are listed as dependencies of the `ukp-platform` package.

Additionally, the `ukp-firewall` package depends on - `python3` - Python 3 runtime - `python3-pyroute2` - Python library for network configuration - `python3-tqdm` - Python library for progress bars (used in TAP pre-opening) - `iptables` - Legacy iptables for firewall rules management - `ipset` - For managing IP sets used in firewall rules - `iproute2` - For network interface management and IP configuration - `net-tools` - For network utilities (e.g., `ifconfig`, `route`) - `bridge-utils` - For managing Linux bridges (if used in custom setups) - `nftables` - For nftables set management by ukpd

12.3 Network Isolation

The platform controls how VMs communicate with each other and the outside world through network isolation.

Isolation Mode

The isolation mode is configured via `--net-mode` or `NET_MODE` in `/etc/ukp.conf`. Currently, only `isolates` is supported:

Mode	Description
isolates	Each VM gets an isolated /30 network. Inter-VM traffic is possible but disabled by default.

Note: Earlier platform versions supported a `subnets` mode, which has been removed.

Internal Traffic Control

The `--net-internal-traffic` flag controls east-west traffic between VMs. This restricts whether VMs can communicate with each other on the internal network.

Accepts a comma-separated list of the following values:

Value	Behavior
<code>srvgrp</code>	Enables internal-to-service-group traffic. VMs can reach service groups belonging to the same user via internal DNS names. The proxy verifies source and destination belong to the same user. All ports of the backing VMs can be reached. Target VMs are woken up from scale-to-zero or scaled-out in the same way as external traffic would do.
<code>vm</code>	Enables VM-to-VM traffic at the network level. When enabled, <code>ukpd</code> populates <code>nftables</code> sets that allow same-user VM-to-VM communication.

(*empty*, No internal traffic allowed.

de-

fault)

Security Implications

With `isolates` mode:

- VMs are placed in separate /30 networks and cannot communicate by default
- Direct inter-VM communication can be enabled for VMs of the same user via `--net-internal-traffic=vm`; all IP traffic between these VMs is then allowed
- Internal VM-to-service-group communication can be enabled via `--net-internal-traffic=srvgrp`; traffic goes through the proxy with all ports reachable (TCP and UDP only)
- This provides strong network isolation between tenants

12.4 TAP Interfaces

Each VM on the platform gets a dedicated TAP network interface that connects it to the host network. The `ukp-vnet.service` pre-provisions these TAP devices and registers them with the firewall ipsets.

`ukp-vnet.service`

The `ukp-vnet.service` is a oneshot `systemd` service that creates and configures TAP devices for VM networking. It runs after `ukp-firewall.service` and must complete before `ukpd` starts.

Command	Behavior
<code>systemctl start ukp-vnet</code>	Creates all TAP devices based on <code>NET_COUNT</code> , assigns IP addresses, and registers each device with the firewall ipsets.
<code>systemctl stop ukp-vnet</code>	Removes all TAP devices and unregisters them from the firewall ipsets.
<code>systemctl restart ukp-vnet</code>	Calls stop then start. All TAP devices are destroyed and recreated.
<code>systemctl reload ukp-vnet</code>	Re-applies TAP device configuration without destroying existing devices. Useful for adding newly provisioned interfaces.

Startup Sequence

When `ukp-vnet.service` starts, it performs the following steps for each network (up to `NET_COUNT`):

1. **Create TAP device** — Creates a TAP device named `ukp<netid>.vif1` using the `ukpvif` utility.
2. **Assign IP address** — Configures the host-side IP address (gateway) on the TAP device with a /30 prefix.
3. **Configure ARP** — Adds a static ARP entry for the guest IP address.
4. **Disable IPv6** — Disables IPv6 on the TAP interface (guest networks are IPv4 only).
5. **Register with ipsets** — Adds the guest network to `ukp.net4` and the gateway address to `ukp.gw4`.

The service uses parallel processing to create TAP devices efficiently. For large deployments (more than 10,000 interfaces), later interfaces are created serially to avoid overwhelming the system.

Configuration

The service reads configuration from `/etc/ukp.conf`:

Variable	Purpose
NET_SEGMENT	The private network segment for VM addresses (10.0.0.0/8, 172.16.0.0/12, or 192.168.0.0/16).
NET_COUNT	Number of TAP devices to create. Each device supports one VM.
NET_MODE	Must be isolates.

Network Addressing

In isolates mode, each TAP device gets its own /30 network carved from NET_SEGMENT:

Address	Purpose
.0	Network address
.1	Guest IP (assigned to the VM)
.2	Gateway IP (host side, assigned to TAP device)
.3	Broadcast address

For example, with NET_SEGMENT=10.0.0.0/8 and NET_COUNT=256, the service creates: - ukp0.vif1 with gateway 10.0.0.2/30 for guest 10.0.0.1 - ukp1.vif1 with gateway 10.0.0.6/30 for guest 10.0.0.5 - ... and so on up to ukp255.vif1

ukpd and TAP device lifecycle

When a VM starts, ukpd allocates a TAP device from the pool managed by ukp-vnet.service, assigns it to the VM, and configures the guest network settings (IP, gateway, DNS). When the VM stops, the TAP device is released back to the pool after a retention period to prevent IP reuse conflicts.

When a VM starts, ukpd:

1. Allocates a TAP device from the interface pool
2. Assigns an IP address from the configured network segment
3. Attaches the TAP device to the VM via Firecracker's network configuration
4. Configures the guest network via startdata (IP, gateway, DNS)

When a VM stops, the TAP device is returned to the pool after a retention period.

Configuration

Flag	Default	Description
--net-iface-preopen	0	Pre-open TAP interfaces at boot. When enabled, ukpd creates TAP devices during startup rather than on-demand at VM creation.

Flag	Default	Description
<code>--net-iface-retention-time-s</code>	15	How long (seconds) a released TAP device is held before being reused. Prevents IP address reuse conflicts.

Pre-Opening TAP Devices

`--net-iface-preopen` is a **boolean** (0 or 1), registered as `OPT_BOOLEAN`.

When enabled (1), `ukpd` pre-opens **all** TAP devices at startup in a background worker thread using lock-free CAS. This removes TAP device creation from the VM start hot path, reducing start latency.

VMs that start before their TAP device is ready (i.e., the background worker hasn't reached it yet) will open the TAP device inline as a fallback.

When disabled (0, the default), TAP devices are created on-demand when each VM starts.

Network Segment

The network segment for VM IP allocation is configured via: `NET_SEGMENT` in `/etc/ukp.conf` (default: `172.16.0.0/12`) - `--net-ip` flag - Allowed segments are `172.16.0.0/12`, `192.168.0.0/16`, and `10.0.0.0/8`

The segment determines the pool of IP addresses available for VM TAP interfaces. Ensure it does not conflict with addresses already in use for other purposes.

Interface Database (IFACEDB)

TAP device allocations are persisted in `ifacedb_*.bin` files in the workspace. Each entry is 256 bytes and tracks the device name, assigned IP, associated VM, and state.

12.5 DNS Services

The platform provides DNS resolution for VMs through three subsystems: `iDNS` (internal), `eDNS` (external), and `coreDNS`.

Architecture

```
VM (guest) —DNS query—> coreDNS (:53)
                        |—> ukpdns plugin —> iDNS (ukpd) [internal names, via Unix socket]
                        |—> forward —> 1.1.1.1 [external names, passthrough]
```

`eDNS` blacklist is enforced separately within `ukpd`, not through `coreDNS`.

iDNS (Internal DNS)

iDNS runs inside ukpd and handles internal VM name resolution. It maps VM names and private FQDNs to their internal IP addresses, allowing VMs to reach each other by name.

Important: iDNS does **not** use standard DNS protocol. It communicates over a Unix domain socket (SOCK_STREAM) at `/var/run/ukp/ukpd-idns.api` using a custom binary protocol.

Protocol Details

The binary protocol defines two message types:

Type	Purpose
SRV	Client identification
STD	Name query

A query consists of the client sending `{client_ipv4, name[]}`, and the server responding with `{n_ipv4, ipv4[]}`.

Records are scoped by user — the source IP is used to determine the user via the platform's internal interface mapping.

Record Types

Type	Description
VM	Maps a hostname to the VM's IP address
SRVGRP	Maps a hostname to the set of IPs belonging to a service group

Registration Lifecycle

- **On VM start:** The platform registers a DNS record for the VM, creating an FQDN from the VM's hostname and the configured `idns-domain`, then inserts it into the hashtable.
- **On VM stop:** The record is removed from the hashtable.

Configuration

Flag	Description
<code>--net-idns-api-endpoint</code>	API endpoint for iDNS queries
<code>--net-idns-domain</code>	Internal DNS domain
<code>--net-idns-ns</code>	Nameserver address for iDNS
<code>--net-idns-proxy-ip</code>	Proxy IP exposed to iDNS

Flag	Description
------	-------------

The iDNS domain is also configurable via `IDNS_ZONE` and `IDNS_HOSTNAME` in `/etc/ukp.conf`.

Note: User-facing DNS name configuration (instance and service FQDNs) is covered in the [public documentation](#). This section covers the infrastructure-level DNS setup relevant to deployments.

eDNS (External DNS)

eDNS runs inside `ukpd` and controls which external domains VMs can resolve. It implements a pattern-matching domain blacklist that filters outbound DNS queries.

Flag	Description
<code>--net-edns-domain</code>	External DNS domain
<code>--net-edns-blacklist</code>	Path to the domain blacklist file

The blacklist file contains domain patterns that VMs are not allowed to resolve.

Blacklist File Format

- One entry per line
- Lines starting with `#` are comments
- Entry format: `<flags>:<pattern>`

Flags:

Flag	Meaning
<code>r</code>	Pattern is a PCRE2 regular expression (JIT-compiled for performance)
<code>g</code>	Rule applies globally only (not per-user)

If no flags are specified, the pattern is treated as an **exact string match**.

If a pattern does not end with `.`, the configured eDNS domain is automatically appended.

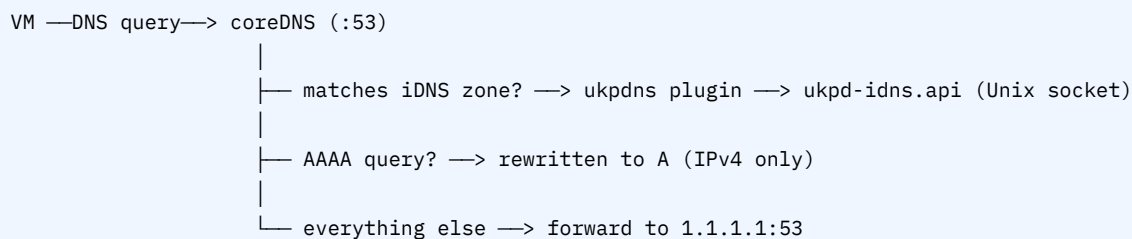
Matching order: The blacklist first performs a hashtable lookup for exact matches, then falls back to sequential PCRE2 evaluation for regex entries.

coreDNS

coreDNS is a standalone DNS server packaged as `ukp-coredns`. It is a **fork of upstream coreDNS** with a single custom plugin (`ukpdns`) that bridges standard DNS queries to `ukpd`'s `iDNS` daemon.

Important architectural note: coreDNS handles **iDNS only** (internal name resolution). It does **not** integrate with eDNS — the eDNS blacklist is enforced separately within `ukpd`. External DNS queries that pass through coreDNS are forwarded directly to Cloudflare (1.1.1.1).

Query Flow



Corefile Configuration

The configuration template is at `/usr/lib/ukp/coredns/ukpdns.conf.template`:

```
.:53 {
    ukpdns ${UKP_IDNS_ZONE}. {
        endpoint ${UKP_IDNS_ENDPOINT}
        mbox ${UKP_IDNS_EMAIL}
        ns ${UKP_IDNS_HOSTNAME}.
        whitelist ${UKP_NET_SEGMENT}
    }

    rewrite stop type AAAA A

    forward . 1.1.1.1:53
}
```

At startup, the launcher script at `/usr/lib/ukp/coredns/launcher/coredns` sources `/etc/ukp.conf`, runs `envsubst` to render the template, and writes the final Corefile to `${UKP_RUNTIME}/coredns/ukpdns.conf`.

ukpdns Plugin Directives

Directive	Value	Purpose
<code>endpoint</code>	<code>\${UKP_RUNTIME}/ukpd-idns.api</code>	Unix socket path to the <code>iDNS</code> daemon
<code>mbox</code>	<code>\${IDNS_EMAIL}</code>	SOA record mailbox field

Directive	Value	Purpose
ns	<code>\${IDNS_HOSTNAME}.</code>	Nameserver hostname for SOA/NS records
whitelist	<code>\${NET_SEGMENT}</code>	Only clients from this CIDR can query the iDNS zone

Compiled Plugins

The `ukp-coredns` package includes only 9 of 60+ available coreDNS plugins:

Plugin	Purpose
log	Query logging
errors	Error logging
prometheus	Metrics endpoint (compiled but not enabled in default config)
rewrite	AAAA→A rewriting
template	DNS response templating
hosts	Static host entries
ukpdns	iDNS bridge (Unikraft custom)
cache	DNS response caching
forward	Upstream DNS forwarding (Cloudflare 1.1.1.1)

All other plugins (kubernetes, dnssec, TLS, gRPC, zone transfer, etc.) are stripped from the build.

Operator Customization

The Corefile template can be customized by editing `/usr/lib/ukp/coredns/ukpdns.conf.template` (or the rendered file at `${UKP_RUNTIME}/coredns/ukpdns.conf`). Possible customizations include:

- Changing the upstream forwarder from 1.1.1.1 to a corporate DNS server
- Enabling the prometheus plugin for DNS metrics
- Adding hosts entries for static name resolution
- Enabling log for query debugging

Systemd Integration

```
ukp-pre.target → ukp-platform.service → ukp-vnet.service → ukp-coredns.service → ukp.target
```

The coreDNS service starts after `ukpd` and the virtual network are ready. It disables `systemd-resolved` on startup to avoid port 53 conflicts.

Dependencies

The `ukp-coredns` package depends on `ukp-config` (`>= 5:1.0`) and `ukp-platform`. It conflicts with upstream `coredns` and `systemd-resolved`.

DNS Configuration in `/etc/ukp.conf`

Variable	Description
<code>DNS_HOSTNAME</code>	Metro node hostname (e.g., <code>fra0</code>)
<code>DNS_ZONE_APP</code>	Application DNS zone (e.g., <code>unikraft.app</code>)
<code>IDNS_ZONE</code>	Internal DNS zone
<code>IDNS_HOSTNAME</code>	Internal DNS hostname

13 Volumes

Volumes provide persistent storage for application instances running on the platform. Unlike the ephemeral root filesystem that is rebuilt from the image on every cold boot, a volume preserves its contents across instance restarts, scale-to-zero cycles, and reattachments to different instances. Volumes are the mechanism for any workload that needs to persist data — databases, file uploads, application state, caches — and they integrate directly with the platform's quota system to enforce per-user and system-wide storage limits.

13.1 Overview

A volume is a persistent storage device backed by SSD. Volumes are managed through VOLDB (`voldb.bin`) and the external `ukp-vol` tool for filesystem operations.

Key characteristics:

- Persistent across instance restarts
- Can be attached to one instance at a time (read-write) or shared across multiple instances (read-only)
- Support ext4 and virtiofs filesystem types
- Support static and dynamic quotas

13.2 Filesystem Types

Volumes support two filesystem types, configured per-volume via the API or globally via `--stor-vol-def-fs`:

Filesystem	Description
ext4	Traditional block-based filesystem. The volume appears as a block device inside the guest.
virtiofs	Host-managed filesystem shared with the guest via the virtio-fs protocol. Supports live resize.

The default filesystem is configurable via `--stor-vol-def-fs` (default: `ext4`).

13.3 Quotas

Volumes support two quota enforcement policies, selected per-volume via the `quota_policy` API field.

Quota policies (`static/dynamic`) are configured per-volume via the API. See the [public documentation](#) for API details.

Static Quotas

- The full `size_mb` is reserved from the user's storage allocation **upfront** — no overcommit.
- Files inside the volume are owned by the configured `squota_uid/squota_gid`.
- Enforcement: filesystem-level UID/GID quotas. When the quota is exceeded, write operations return `-EDQUOT`.

Dynamic Quotas

Warning: If a dynamic quota volume exceeds its limit, the filesystem may become corrupted. Monitor volume usage and set appropriate limits.

- Uses Linux **project quotas** (`PRJQUOTA`). The project ID is calculated as `10 + volume_internal_id`.
- Also uses **user quotas** (`USRQUOTA`) for per-user storage totals and **group quotas** (`GRPQUOTA`) for system-wide totals.
- Supported on **ext4 and XFS only**.
- Enforcement: filesystem hard limit. If the quota is exceeded, writes may fail and can **corrupt an embedded filesystem** (e.g., an ext4 filesystem inside a virtiofs volume).

Configuration Flags

Flag	Description
<code>--stor-squota-uid</code>	UID for static quota enforcement
<code>--stor-squota-gid</code>	GID for static quota enforcement
<code>--stor-dquota-gid</code>	GID for dynamic quota enforcement

13.4 Storage Operations

Discard Control

The `--stor-vol-disable-discard` flag controls whether TRIM/discard operations are passed through to the underlying storage. Disabling discard can improve performance on some storage backends at the cost of not reclaiming freed blocks.

Parallel Operations

The `--stor-max-async-ops` flag (default: 16) controls the maximum number of concurrent filesystem operations. This affects volume creation, deletion, cloning, and formatting throughput.

Increasing this value can improve performance on systems with fast storage, but may cause I/O saturation on slower backends.

Live Resize

Live resize of virtiofs volumes is documented in the [public documentation](#).

13.5 Volume Storage

Volume data is stored at the path configured via `--stor-vol-path`. The `ukp-vol` tool (configured via `--stor-vol-tool`) handles filesystem-level operations: creation, deletion, cloning, and formatting.

Filesystem configuration templates are installed to `/usr/lib/ukp/platform/config/fs/`.

13.6 Volume Limits

Per-user volume limits are configurable:

Flag	Description
<code>--stor-user-max-volumes</code>	Maximum number of volumes per user
<code>--stor-max-volumes</code>	System-wide maximum number of volumes
<code>--stor-user-min-volume-mb</code>	Minimum volume size per user (MB)
<code>--stor-user-max-volume-mb</code>	Maximum volume size per user (MB)
<code>--stor-user-max-total-volume-mb</code>	Total volume storage budget per user (MB)
<code>--stor-max-total-volume-mb</code>	Total volume storage budget across all users (MB)

13.7 Managed Volumes

Managed volumes are volumes whose lifecycle and storage backend are fully managed by the platform. Contact Unikraft for guidance on configuring managed volume support for your deployment.

13.8 Custom Filesystem Configuration

The platform uses filesystem configuration templates to manage volume lifecycle operations (formatting, mounting, unmounting, and deleting). These templates are stored at the path configured by `--stor-fs-config-path` (default: `/usr/lib/ukp/platform/config/fs/`).

Operators can add custom filesystem types by creating configuration scripts in this directory. Each script defines how the platform handles a specific filesystem type through its lifecycle:

- **Formatting** — How to create and initialize the filesystem on a new volume
- **Mounting** — How to mount the filesystem when attaching a volume to an instance
- **Unmounting** — How to cleanly unmount the filesystem when detaching
- **Deleting** — How to clean up the filesystem when a volume is destroyed

This enables advanced use cases such as mounting custom remote storage (e.g., network-attached or distributed storage backends) on volume mount, or integrating with site-specific storage infrastructure.

Note: Contact Unikraft for guidance on configuring custom filesystems for your deployment.

14 Images

Images are the deployable units on the platform — each image contains a kernel and a root filesystem packaged as an OCI artifact. Before an instance can run, its image must be present on the host, this is the job of the platform agent.

For air-gapped deployments the platform supports importing static images.

This chapter covers image storage, distribution, and management from the operator's perspective.

14.1 Image Storage

Images are stored on disk at the path configured via `UKP_IMAGES`. The on-disk layout is:

```
/var/lib/ukp/images/unikraft.io/<org>/<image>/
  @<digest>/unikraft/bin/kernel
  @<digest>/unikraft/bin/initrd
  @<digest>/config.json
  @<digest>/cmdline.txt
```

14.2 Image Distribution Modes

See [Agent: Image Management](#) for the three registry modes (external+webhook, external+on-demand, embedded).

14.3 Offline Image Import

The `--images-import-path` flag specifies a **JSON file** (not a directory) from which `ukpd` imports images at startup. This enables deployments where the host has no network access to an image registry.

Import File Format

The JSON file contains an array of image objects:

```
[
  {
    "owner": "550e8400-e29b-41d4-a716-446655440000",
    "url": "https://example.com/image@sha256:abc123...",
    "config": "org/image/digest/config.json",
    "kernel": "org/image/digest/unikraft/bin/kernel",
    "initrd": "org/image/digest/unikraft/bin/initrd"
  }
]
```

```
]
```

Field	Description
owner	Optional UUID of the owning user. If empty, the image is public.
url	Image URL (used as the image reference)
config	Path to config.json, relative to UKP_IMAGES
kernel	Path to kernel binary, relative to UKP_IMAGES
initrd	Path to initrd, relative to UKP_IMAGES
rom	Path to ROM file (alternative to initrd), relative to UKP_IMAGES

Images are loaded at startup. Duplicates are skipped, and imported images are marked as **PERSISTENT** (not subject to garbage collection).

14.4 Image References

Note: Image references (URL, tag, digest formats) are also covered in the [public documentation](#). This section covers operator-level image management.

Images can be referenced in multiple ways:

By tag or digest

The standard OCI reference format: `<org>/<image>:<tag>` or `<org>/<image>@sha256:<digest>`.

By URL

Images can be specified as a URL string pointing to the image location. See the notes about the HTTP image transport under [Agent: Image Management](#).

ImageSpec Format

Images can be specified in two forms in the API:

Form 1 — Simple string:

```
{
  "image": "https://example.com/image@sha256:abc123..."
}
```

Form 2 — Object with headers:

```
{
  "image": {
    "url": "https://example.com/image@sha256:abc123...",
    "headers": {
      "Authorization": "Bearer token"
    }
  }
}
```

- A maximum of **16 headers** can be specified.
- The URL must be **content-addressable** — the same URL must always return the same image content.

15 TLS/SSL Certificates Management

The platform requires TLS certificates for securing API endpoints and application traffic. Certificate management is handled by the `ukp-lego` package, which supports both automatic Let's Encrypt certificates via ACME DNS-01 challenge and self-signed certificates for air-gapped or development environments.

15.1 Overview

The platform uses three certificates:

Certificate	Domain Pattern	Purpose
API	<code>api.<hostname>.<zone></code>	Platform API endpoint (e.g., <code>api.fra0.unikraft.cloud</code>)
Index	<code>index.<hostname>.<zone></code>	Instance index endpoint (e.g., <code>index.fra0.unikraft.cloud</code>)
Wildcard	<code>*.<hostname>.<zone></code>	Application instances (e.g., <code>*.fra0.unikraft.app</code>)

Certificates are stored in `${UKP_DATA}/lego/certificates/` (typically `/var/lib/ukp/data/lego/certificates/`) and symlinked to `${UKP_DATA}/certificates/` for access by OpenResty.

15.2 Certificate Modes

Let's Encrypt Certificates

When DNS provider credentials are configured, the platform uses Let's Encrypt to issue trusted certificates automatically. The ACME DNS-01 challenge is used, which requires API access to the DNS provider.

Configuration is provided through an environment file specified by `LEGO_ENVFILE` in `/etc/ukp.conf`, or via the legacy `CLOUDFLARE_DNS_EMAIL` and `CLOUDFLARE_DNS_API_TOKEN` variables for Cloudflare.

Environment File Format

The environment file must define:

Variable	Description
<code>LEGO_DNS_PROVIDER</code>	DNS provider name (e.g., <code>cloudflare</code> , <code>route53</code> , <code>gcloud</code>). See Lego documentation for supported providers.
<code>LEGO_CERT_EMAIL</code>	Email address for Let's Encrypt account registration and expiry notifications.

Variable	Description
Provider-specific variables	Credentials for the DNS provider (e.g., <code>CF_DNS_API_TOKEN</code> for Cloudflare).

Example environment file for Cloudflare (`/etc/ukp.d/lego.d/envfile`):

```
LEGO_DNS_PROVIDER=cloudflare
LEGO_CERT_EMAIL="admin@example.com"
CF_API_EMAIL="admin@example.com"
CF_DNS_API_TOKEN="your-cloudflare-api-token"
```

Then configure `/etc/ukp.conf`:

```
LEGO_ENVFILE="/etc/ukp.d/lego.d/envfile"
```

Legacy Cloudflare Configuration

For backwards compatibility, Cloudflare credentials can be specified directly in `/etc/ukp.conf`:

```
CLOUDFLARE_DNS_EMAIL="admin@example.com"
CLOUDFLARE_DNS_API_TOKEN="your-cloudflare-api-token"
```

This generates an internal environment file automatically. For other DNS providers, use `LEGO_ENVFILE`.

Self-Signed Certificates

When no DNS provider credentials are configured, the platform falls back to self-signed certificates. These are valid for 10 years and are suitable for:

- Development and testing environments
- Air-gapped deployments without internet access
- Internal deployments where external trust is not required

Self-signed certificates trigger browser warnings and are not trusted by default. For production deployments, configure Let's Encrypt certificates. If the platform is configured to self-sign mode, the `update-ukp-certificates` script will generate the required certificates at startup if they do not already exist. If they exist, they will be reused to avoid unnecessary regeneration. However, the files are also not checked for renewal and kept as is. Renewal of self-signed certificates can be enforced by running `update-ukp-certificates` with the `-f` flag to force regeneration.

Operator-Managed Certificates

For environments with existing PKI infrastructure or specific compliance requirements, operators can provide their own certificates. This approach gives full control over certificate issuance, validity periods, and renewal schedules.

Setup

1. **Disable automatic certificate management** — Remove or comment out Lego credentials in `/etc/ukp.conf`, this is identical to switching to self-signed mode:

```
# LEGO_ENVFILE=""  
# CLOUDFLARE_DNS_EMAIL=""  
# CLOUDFLARE_DNS_API_TOKEN=""
```

2. Alternatively, uninstall the `ukp-lego` package entirely if automatic certificate management is not needed:

```
apt remove ukp-lego
```

3. **Create the certificates directory** — Ensure the directory exists:

```
mkdir -p /var/lib/ukp/data/lego/certificates
```

4. **Copy certificates** — Place certificates and private keys in the certificates directory using the expected naming convention:

Certificate	Certificate File	Key File
API	<code>api.<hostname>.<DNS_ZONE_APP>.<hostname>.<DNS_ZONE_API>.cert</code>	<code>api.<hostname>.<DNS_ZONE_API>.key</code>
Index	<code>index.<hostname>.<DNS_ZONE_APP>.<hostname>.<DNS_ZONE_API>.cert</code>	<code>index.<hostname>.<DNS_ZONE_API>.key</code>
Wildcard	<code>_.<hostname>.<DNS_ZONE_APP>.<hostname>.<DNS_ZONE_API>.cert</code>	<code>_.<hostname>.<DNS_ZONE_API>.key</code>

For example, with `DNS_HOSTNAME=fra0`, `DNS_ZONE_API=unikraft.cloud`, and `DNS_ZONE_APP=unikraft.app`:

```
cp /path/to/api.cert /var/lib/ukp/data/lego/certificates/api.fra.unikraft.cloud.cert  
cp /path/to/api.key /var/lib/ukp/data/lego/certificates/api.fra.unikraft.cloud.key  
cp /path/to/index.cert /var/lib/ukp/data/lego/certificates/index.fra.unikraft.cloud.cert  
cp /path/to/index.key /var/lib/ukp/data/lego/certificates/index.fra.unikraft.cloud.key  
cp /path/to/wildcard.cert /var/lib/ukp/data/lego/certificates/_.fra.unikraft.app.cert  
cp /path/to/wildcard.key /var/lib/ukp/data/lego/certificates/_.fra.unikraft.app.key
```

5. **Create PEM files** — OpenResty requires combined PEM files containing both certificate and key:

```
cd /var/lib/ukp/data/lego/certificates
cat api.fra.unikraft.cloud.crt api.fra.unikraft.cloud.key > api.fra.unikraft.cloud.pem
cat index.fra.unikraft.cloud.crt index.fra.unikraft.cloud.key >
  ↪ index.fra.unikraft.cloud.pem
cat _.fra.unikraft.app.crt _.fra.unikraft.app.key > _.fra.unikraft.app.pem
```

6. Set permissions — Ensure proper ownership and permissions:

```
chown root:root /var/lib/ukp/data/lego/certificates/*
chmod 600 /var/lib/ukp/data/lego/certificates/*.key
chmod 600 /var/lib/ukp/data/lego/certificates/*.pem
chmod 644 /var/lib/ukp/data/lego/certificates/*.crt
```

7. Reload OpenResty — Apply the new certificates:

```
systemctl reload ukp-openresty
```

Certificate Requirements

Operator-provided certificates must meet the following requirements:

- **Format:** PEM-encoded X.509 certificates
- **Key type:** RSA (2048-bit minimum) or ECDSA (P-256 or P-384)
- **Wildcard certificate:** Must include both `*.<hostname>.<DNS_ZONE_APP>` and `<hostname>.<DNS_ZONE_APP>` as Subject Alternative Names

Renewal Responsibility

With operator-managed certificates, renewal is the operator's responsibility. Before certificates expire:

1. Obtain renewed certificates from your CA.
2. Replace the certificate and key files in `/var/lib/ukp/data/lego/certificates/`.
3. Regenerate the PEM files.
4. Reload OpenResty:

```
systemctl reload ukp-openresty
```

Consider setting up monitoring alerts for certificate expiration to avoid service disruptions.

15.3 update-ukp-certificates

The `update-ukp-certificates` script manages certificate issuance and renewal. It is installed to `/usr/lib/ukp/lego/bin/update-ukp-certificates`.

Usage

```
update-ukp-certificates [OPTIONS]
```

Option	Description
-h	Display help and exit.
-f	Force mode: Delete and re-issue existing certificates. Required when switching between self-signed and Let's Encrypt modes.

Behavior

When invoked, the script:

1. **Loads configuration** — Reads `/etc/ukp.conf` for DNS settings and credentials.
2. **Detects mode** — Determines whether to use Let's Encrypt or self-signed based on credential availability.
3. **Issues certificates** — For each required certificate:
 - If the certificate does not exist, issues a new one.
 - If using Let's Encrypt and the certificate exists, checks if renewal is needed (within 10 days of expiry).
 - If using self-signed and the certificate exists, skips (unless `-f` is specified).
4. **Runs renewal hook** — After issuing or renewing, creates a combined PEM file and reloads OpenResty.

Issued Certificates

The script issues certificates for:

Domain	Files
api.<hostname>.<DNS_ZONE_API>	api.<hostname>.<DNS_ZONE_API>.cert, api.<hostname>.<DNS_ZONE_API>.key
index.<hostname>.<DNS_ZONE_API>	index.<hostname>.<DNS_ZONE_API>.cert, index.<hostname>.<DNS_ZONE_API>.key
*.<hostname>.<DNS_ZONE_APP>	_.<hostname>.<DNS_ZONE_APP>.cert, _.<hostname>.<DNS_ZONE_APP>.key

The wildcard certificate also includes the bare domain (`<hostname>.<DNS_ZONE_APP>`) as a Subject Alternative Name (SAN).

Manual Invocation

Run manually to issue or renew certificates immediately:

```
# Issue or renew certificates (normal operation)
/usr/lib/ukp/lego/bin/update-ukp-certificates

# Force re-issue all certificates (e.g., after switching modes)
/usr/lib/ukp/lego/bin/update-ukp-certificates -f
```

15.4 Automatic Renewal

The `ukp-lego` package installs a cron job at `/etc/cron.d/ukp-lego` that runs the certificate update script twice weekly:

```
# Refreshes twice a week (2am on Sundays and Wednesdays)
0 2 * * 0,3 root /usr/lib/ukp/lego/bin/update-ukp-certificates > /dev/null 2>&1
```

This schedule ensures certificates are renewed well before expiry. Let's Encrypt certificates are renewed when within 10 days of expiration. Self-signed certificates (or operator-managed certificates) are never automatically renewed.

15.5 Renewal Hook

After a certificate is issued or renewed, the renewal hook at `/usr/lib/ukp/lego/hooks/renew` is executed. The hook:

1. **Creates PEM file** — Combines the certificate and private key into a single `.pem` file for OpenResty.
2. **Reloads OpenResty** — Signals `ukp-openresty.service` to reload and pick up the new certificate.

The hook receives the certificate paths via environment variables:

Variable	Description
LEGO_CERT_PATH	Path to the issued certificate file.
LEGO_CERT_KEY_PATH	Path to the private key file.

15.6 Switching Certificate Modes

When switching between self-signed and Let's Encrypt certificates (or vice versa):

1. Update `/etc/ukp.conf` with the new credentials (or remove them for self-signed).

2. Force re-issue certificates:

```
/usr/lib/ukp/lego/bin/update-ukp-certificates -f
```

3. Verify the new certificates are in place:

```
ls -la /var/lib/ukp/data/lego/certificates/
```

15.7 Certificate Storage

Path	Description
/var/lib/ukp/data/lego/	Lego working directory containing ACME account data and certificates.
/var/lib/ukp/data/lego/cert	Issued certificates and private keys.
/var/lib/ukp/data/certificates	Symlink to the certificates directory, used by OpenResty.
/var/run/ukp/lego/	Temporary directory for runtime state.

16 Metrics

Metrics provide visibility into the health and performance of the platform and the instances running on it. As an operator, metrics are the primary tool for capacity planning, performance tuning, and alerting. The platform exposes metrics at two levels: platform-specific counters, histograms, and per-user gauges from ukpd, and standard system metrics (CPU, memory, disk, network) from the agent's embedded Prometheus node_exporter.

Note: Per-VM metrics and the wakeup latency histogram are available via the API and are documented in the [public documentation](#).

16.1 Architecture

Metrics are collected from platform components and made available through dedicated Prometheus-compatible endpoints:

1. /metrics/controller: Controller metrics
2. /metrics/host: Host-level metrics (embedded node_exporter)
3. /metrics/agent: Agent metrics

The response is in standard Prometheus text exposition format. A valid Bearer token is required for authentication. The token can be configured in /etc/ukp.conf.

In addition, the --vmm-metrics-emit flag of the controller determines whether extended per-VM metrics are emitted by the VMM. When --vmm-metrics-emit=1, the VMM writes a metrics.log file (its native JSON format) in each VM's working directory. These are not exported via any endpoint and primarily for debugging purposes.

16.2 Controller Metrics

The following platform-specific metrics are exposed on the Prometheus endpoint:

Counters

Metric	Type	Description
vmdb_instances_created	counter	Total number of instances created
vmdb_instances_deleted	counter	Total number of instances deleted

Histograms

Metric	Type	Description
<code>vmdb_instances_deleted_uptime</code>	histogram	Uptime of deleted instances. Uses exponential buckets starting at 1000 ms, factor 2×, 16 buckets.

Per-User Gauges

These gauges are labeled by user and reflect current resource usage:

Metric	Type	Description
<code>vmdb_instances</code>	gauge	Number of instances owned by the user
<code>vmm_instances</code>	gauge	Number of running VMM instances for the user
<code>vmm_vcpus</code>	gauge	Total vCPUs in use by the user
<code>vmm_memory_mb</code>	gauge	Total memory (MB) in use by the user
<code>net_service_groups</code>	gauge	Number of service groups owned by the user
<code>net_services</code>	gauge	Number of services owned by the user

16.3 Node-Level Metrics

These metrics are relevant for deployments where operators manage the host infrastructure directly.

The agent's Prometheus `node_exporter` provides standard system metrics:

- CPU, memory, disk, and network utilization
- Filesystem usage
- System load averages

These are standard `node_exporter` metrics and follow the upstream naming conventions (e.g., `node_cpu_seconds_total`, `node_memory_MemAvailable_bytes`).

17 Logging

Logging is the primary mechanism for understanding what the platform is doing at runtime and for diagnosing issues after the fact. The platform supports structured logging to journald (recommended for systemd-based deployments), file-based logging with rotation, and configurable log sinks that can forward events to external systems over Unix sockets or to files in multiple encoding formats. Log sinks are particularly useful for integrating with centralized log aggregation systems or for capturing events for audit purposes.

17.1 Log Format

The `--log-format` flag selects the log output format:

Value	Format	Description
0	Default	Plain text log output
1	Journald	Structured output compatible with systemd-journal

When running under systemd (the default deployment), journald format (1) is recommended. This enables structured queries via `journalctl`:

```
# View ukpd logs
journalctl -u ukp-platform.service

# View agent logs
journalctl -u ukp-agent.service

# Follow logs in real time
journalctl -u ukp-platform.service -f
```

17.2 Log Path and Level

Flag	Description
<code>--log-path</code>	Path to the log file (when not using journald)
<code>--log-level</code>	Log verbosity level
<code>--log-format</code>	Output format (0 = default, 1 = journald)

The `LOG_BACKEND` variable in `/etc/ukp.conf` can also control the logging backend.

17.3 Log Rotation

The `--log-rotation` flag controls how log files are managed when they reach a size threshold or when the process receives a signal.

SIGHUP Log Reopening

Sending SIGHUP to the ukpd process causes it to reopen its log files. This is useful when integrating with external log rotation tools like logrotate:

```
# Example logrotate config for ukpd
/var/log/ukp/platform.log {
    daily
    rotate 7
    compress
    missingok
    postrotate
        kill -HUP $(cat /run/ukp/platform.pid)
    endscript
}
```

17.4 Log Sinks

The platform supports forwarding logs to external sinks via a JSON configuration file. A `sinks-example.json` template is provided with the platform installation.

Multiple sinks can be configured simultaneously. The configuration is a JSON array:

```
[
  {
    "name": "<string>",           // Required: human-readable label
    "type": "<sink_type>",        // Required: "file" or "socket"
    "encoding": "<encoding>",    // Required: "binary", "text", "journald", or "json"
    "path": "<string>",          // Required: file path, Unix socket path, "stdout", or
    ↪ "stderr"
    "events": [<event>, ...],    // Required: event types (see below)
    "reliable": <bool>,          // Optional: persistent buffer (default: false)
    "buffer_size_kb": <int>,     // Optional: must be power of 2, range 1-1048576 (default:
    ↪ 16)
    "backoff_time_s": <int>      // Optional: reconnect backoff 1-3600 (default: 2)
  }
]
```

Sink Types

- **file:** Writes log events to a file at path. The path can also be "stdout" or "stderr" to write to standard output/error.
- **socket:** Connects to a Unix domain socket at path and streams log events.

Event Types

The events array specifies which event types the sink receives:

Event Type	Description
log.debug	Debug-level log messages
log.info	Informational log messages
log.warning	Warning-level log messages
log.error	Error-level log messages
vm.state_change	VM state transitions

Category prefixes can be used as shorthand: - "log" — matches all log events (log.debug, log.info, log.warning, log.error) - "vm" — matches all VM events (vm.state_change)

Encoding Formats

Encoding	Output Format
binary	Raw binary serialization via libserialize
text	[ISO8601] [PREFIX] TEXT
journald	<PRIORITY LEVEL>TEXT (syslog-style priority prefix)
json	{ "timestamp": "ISO8601", "type": "<TYPE>", "data": { ... } }

Buffering

Log sinks use a lock-free MPSC (multi-producer, single-consumer) ring buffer. When `reliable` is set to `true`, the ring buffer is backed by a persistent memory-mapped file, ensuring buffered events survive controller restarts. The `buffer_size_kb` parameter controls the ring buffer capacity.

18 Health Checks

Health checks allow the platform, external load balancers, and monitoring systems to determine whether a host is ready to serve traffic. The platform runs health checks at two levels: the agent monitors the state of platform systemd services and the presence of required images, while ukpd runs an internal health monitor as part of its monitor subsystem. When all checks pass and the node reaches `ukp-online.target`, it is considered healthy. Integrating the health endpoint into your load balancer or monitoring stack ensures that traffic is only routed to hosts that are fully operational.

18.1 Overview

Health checks run at two levels:

1. **Agent health checks** — The agent monitors the health of images, systemd services, and its own subsystems.
2. **ukpd health monitor** — ukpd runs an internal health monitor as part of its monitor subsystem.

When all health checks pass and the node reaches `ukp-online.target`, it is considered healthy and ready to serve traffic.

Health Endpoint

The agent exposes a health endpoint:

```
GET /v1/healthz
```

Healthy response (HTTP 200):

```
{"status": "success", "message": "All internal systems operational."}
```

Degraded response (HTTP 500):

```
{"status": "error", "message": "Degraded."}
```

Simple mode (`--enable-simple-healthz-api`, default: `true`): Returns only the HTTP status code with no JSON body. The simple health endpoint does not require authentication. Consider network-level access controls if the health endpoint should not be publicly accessible.

18.2 Agent Health Checks

The agent runs two health checkers:

Checker	Description
---------	-------------

Systemd checker Monitors platform systemd services via D-Bus: ukp-coredns.service, ukp-openresty.service, ukp-platform.service, ukp-pulld.service. A service is healthy if its ActiveState is active or reloading **and** its SubState is running or auto-restart.

Images checker Verifies that the default images are present on the node. Reports degraded if any required images are missing.

18.3 Configuration

Flag	Default	Description
--health-service-check-interval	10s	How often health checks run
--enable-simple-healthz-api	true	Enable simple mode (HTTP status only, no JSON body, public access)

19 Troubleshooting

When something goes wrong on the platform — instances fail to start, certificates do not provision, or the controller crashes — this chapter is the starting point for diagnosis. It covers the diagnostic tools available for inspecting platform state, the most common issues operators encounter in production, and the steps to resolve them. The platform stores all of its state in binary databases that can be inspected and repaired with the `ukp-db` tool, and all components output logs.

19.1 Diagnostic Tools

`systemctl`

Check the status of platform services:

```
# Check all platform units
systemctl list-units 'ukp-*'

# Check if the platform is online
systemctl is-active ukp-online.target

# Restart ukpd
systemctl restart ukp-platform.service
```

Platform logs

Depending on the `LOG_BACKEND` configuration, logs are stored in files and/or `systemd-journald`. Refer to the following sections for the configuration and location.

Files

This is used if `LOG_BACKEND=file` or is not set. Note that some components always log into the journal instead of writing to a file.

The paths for the individual log files are as follows:

- `ukpd`: `/var/log/ukp/platform/ukpd.log`
- `OpenResty`: `/var/log/ukp/openresty/error.log`

For the other services, refer to the [systemd-journald](#) sections.

`systemd-journald`

This is used if `LOG_BACKEND=journald`. The logs are placed into the `ukp-platform` journal namespace. Some example queries for the logs:

```
# ukpd + openresty logs from the last hour
journalctl --namespace=ukp-platform --since "1 hour ago"

# ukpd logs from the last hour
journalctl --namespace=ukp-platform -u ukp-platform.service --since "1 hour ago"

# Agent logs
journalctl -u ukp-agent.service --since "1 hour ago"

# All platform units
journalctl --namespace=\* -u 'ukp-\*' --since "1 hour ago"

# Follow ukp logs in real time
journalctl -u ukp-platform.service -f
```

ukp-db

The ukp-db tool provides command-line inspection and repair for the platform's internal databases (VMDB, SRVDB, CERTDB, IMGDB, VOLDB, IFACEDB).

```
ukp-db /path/to/workspace           # Open all DBs interactively
ukp-db --db vmdb /path/to/vmdb.bin  # Open specific DB
ukp-db --exec "print(vmdb[0])"       # Execute inline script
ukp-db --exec-file script.py         # Execute script file

# In the interactive shell:
for vm in vmdb: print(vm)           # Iterate entries
vmdb["uuid-here"]                   # UUID lookup
vmdb.filter_active()                # Filter active entries
vmdb.header                         # DB header info
vmdb.rebuild_lists()                # Repair free/zombie lists
```

Import / Export with ukp-db-ix

The ukp-db-ix tool exports and imports VM entries along with their dependencies (volumes, services, certificates, interfaces) as JSON. This is used for migrating VMs between hosts.

```
# Export: write a list of VM UUIDs to uuids.txt, then export
ukp-db-ix export WORKSPACE uuids.txt > out.json  # Export VMs + deps

# Import: load entries from JSON into the target workspace
ukp-db-ix import WORKSPACE in.json              # Import entries
ukp-db-ix import -n WORKSPACE in.json           # Dry run (validate only)

# Inspect an export file
ukp-db-ix list-uuids in.json vmdb              # List exported UUIDs
```

```
ukp-db-ix list-files WORKSPACE in.json snapshots # List snapshot files to copy
```

Migration workflow: 1. ukp-db-ix export on the source host to produce a JSON file. 2. ukp-db-ix list-files ... snapshots to identify snapshot files that need to be copied. 3. Copy the JSON file and snapshot files to the target host. 4. ukp-db-ix import -n on the target host to validate. 5. ukp-db-ix import to commit the entries into the target databases.

19.2 Stop Codes

Stop codes are documented in the [public documentation](#).

19.3 Common Issues

Platform fails to start

Symptoms: ukp-platform.service fails to start, or stays in activating state.

Diagnostic steps: 1. Check logs: journalctl -u ukp-platform.service -n 100 2. Check configuration: Verify /etc/ukp.conf syntax 3. Check disk space: Platform databases require space at the workspace path (--workspace / -w) 4. Check port conflicts: The API endpoint and proxy ports must be available

Agent cannot connect to ukpd

Symptoms: ukp-agent.service repeatedly restarts, logs show connection errors.

Diagnostic steps: 1. Verify ukpd is running: systemctl is-active ukp-platform.service 2. Check the Unix socket: ls -la /run/ukp/platform.sock (or the configured socket path) 3. Check agent configuration: Verify the agent's endpoint configuration

Instances fail to start

Symptoms: Instance creation succeeds but the VM never reaches running state.

Diagnostic steps: 1. Check the VM state: ukp-db vmdb show <vm-id> 2. Check Firecracker logs: Look for the per-VM log file under the configured log path 3. Check image availability: Verify the image exists at /var/lib/ukp/images/ 4. Check resource availability: Verify sufficient memory and CPU

Scale-to-zero instances fail to resume

Symptoms: Requests to a suspended instance time out instead of waking the instance.

Diagnostic steps: 1. Check the VM state in VMDB (should be in standby state) 2. Check that the snapshot files exist 3. Check proxy logs for the wake trigger 4. Check the wakeup_latency metric for timing data

Certificate provisioning fails

Symptoms: HTTPS traffic fails, certificates show as pending.

Diagnostic steps: 1. Check agent logs for ACME errors: `journalctl -u ukp-agent.service | grep -i acme` 2. Verify DNS resolution for the domain 3. Check CERTDB state: `ukp-db certdb list` 4. Verify network connectivity to the ACME provider (Let's Encrypt)



Unikraft Cloud Platform Enterprise Documentation

Copyright © 2026 Unikraft GmbH.

All rights reserved.

PRIVATE AND CONFIDENTIAL

unikraft.com